

DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 1:

Advance Sequential Logic

Overview

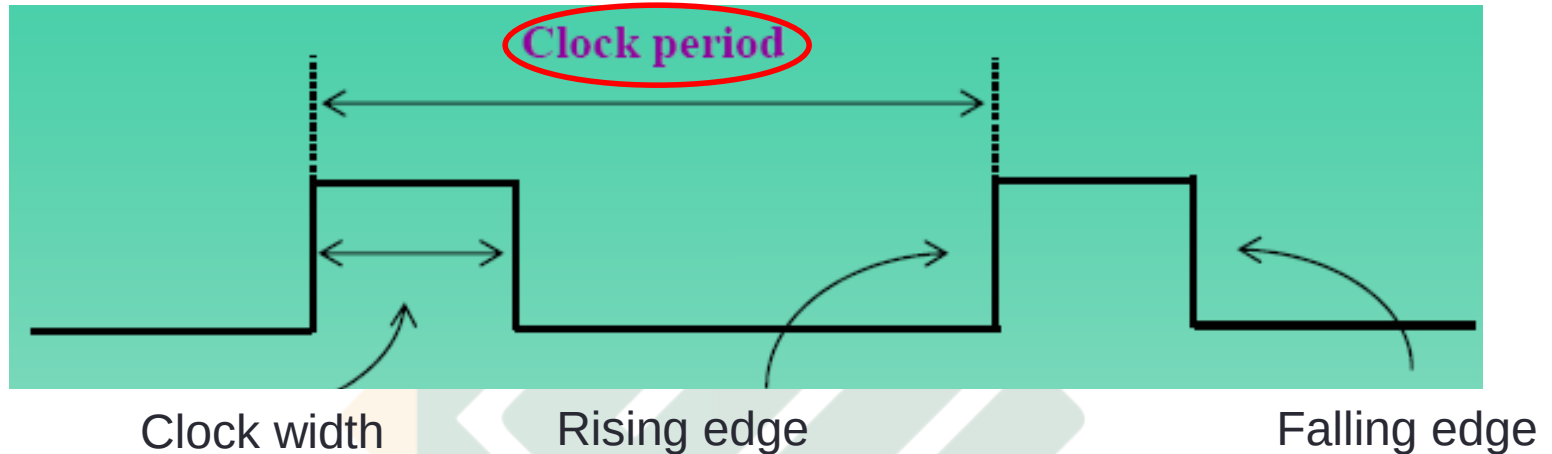
- Latches and Flip-flops
- Basic components of Memory
- **Finite State Machine (FSM)**
- **Analyse the Sequential Circuit**
- **Design the Sequential Circuit**
- **Synthesis the Sequential Circuit**

COMPUTER ENGINEERING

Sequential components

- Sequential components contain **memory** elements
- The output values of sequential components depend on the input values and/or the values stored in the memory elements
- The values in the memory elements define the state of sequential components.
- Example : Ring counter that starts the answering machine after 4 rings
- Sequential components can be: asynchronous or synchronous.
 - **Asynchronous** sequential components change their state and output values as a response to change in input values.
 - **Synchronous** sequential components change their state and output values *at fixed* points of time defined by the clock signal.

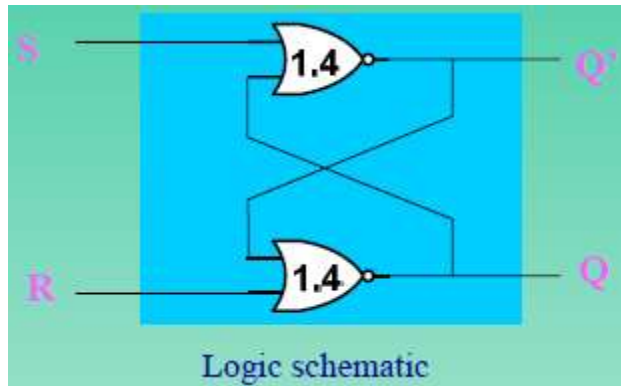
Clock signal



- **Clock period** (measured in **micro-** or **nano-seconds**) is the time between successive transitions in the same direction
- **Clock frequency** (measured in **MHz** or **GHz**) is the reciprocal of clock period
- **Clock width** is the time interval during which clock is equal to 1
- **Duty cycle** is the ratio of the clock width and clock period

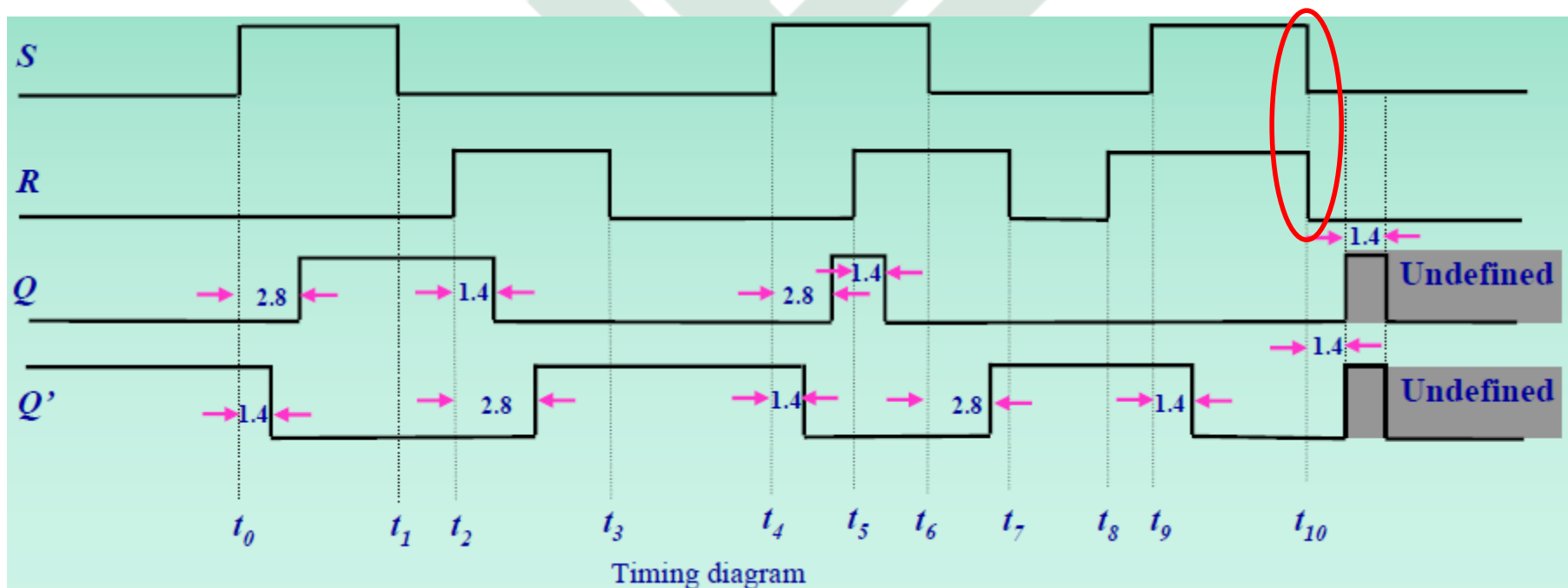
SR - latch (NOR implementation)

SR-latch has two states: **set** state ($Q=1$) and **reset** state ($Q=0$).

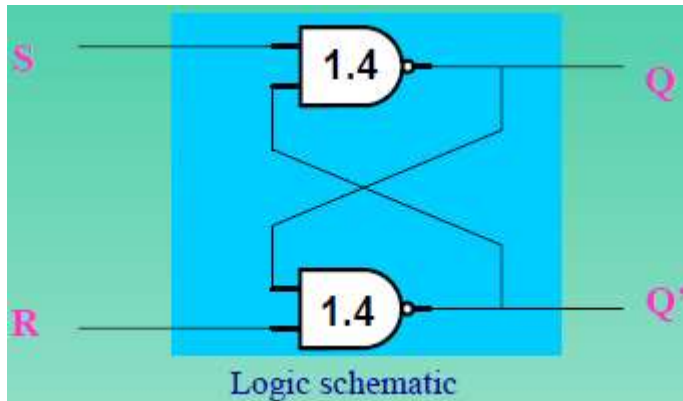


S	R	Q	$Q (next)$	$Q' (next)$	
0	0	0	0	1	(hold)
0	0	1	1	0	(hold)
0	1	X	0	1	(reset)
1	0	X	1	0	(set)
1	1	X	0	0	(?)

Truth table

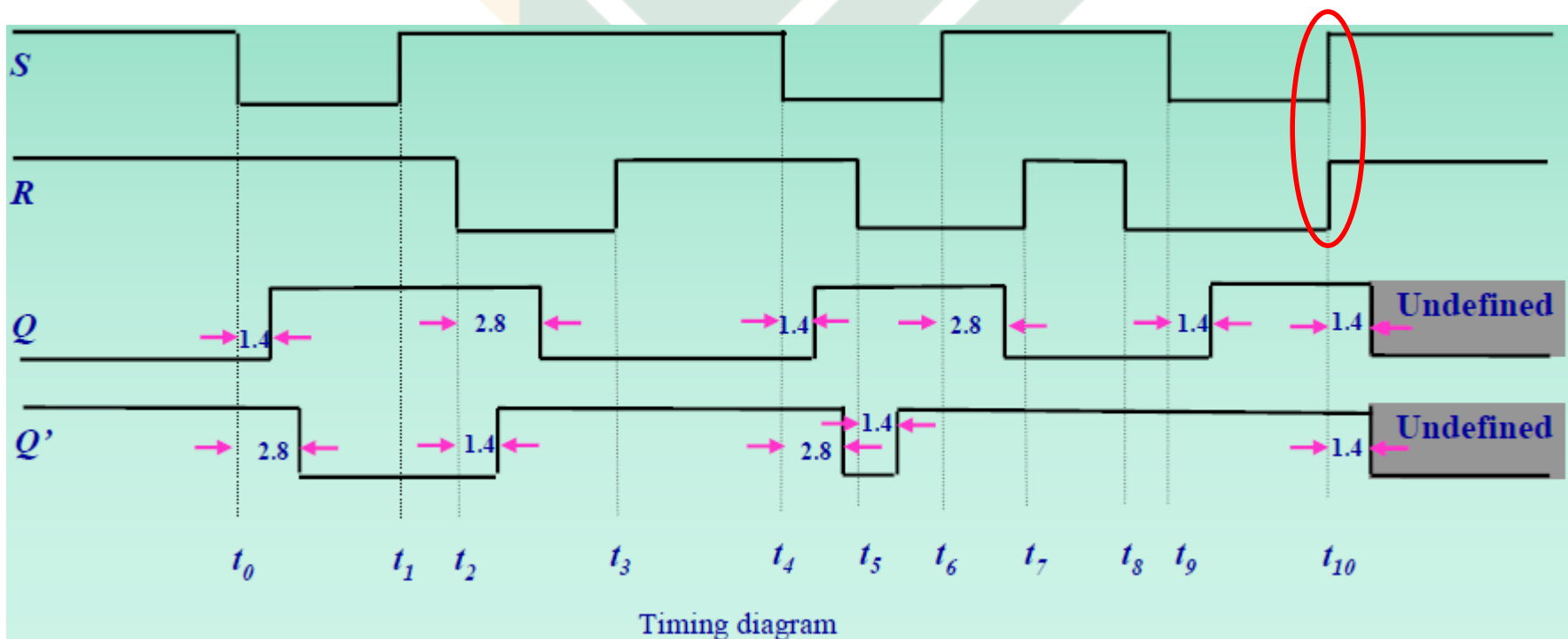


SR - latch (NAND implementation)



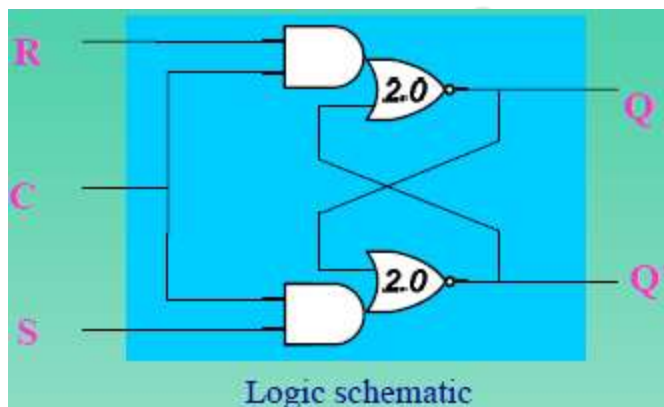
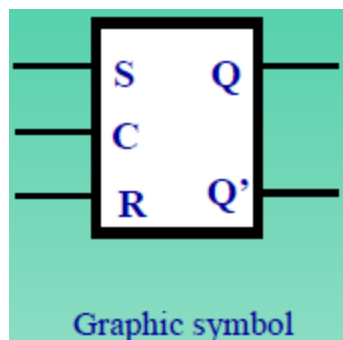
<i>S</i>	<i>R</i>	<i>Q</i>	<i>Q</i> (next)	<i>Q'</i> (next)	
0	0	X	1	1	(?)
0	1	X	1	0	(set)
1	0	X	0	1	(reset)
1	1	0	0	1	(hold)
1	1	1	1	0	(hold)

Truth table



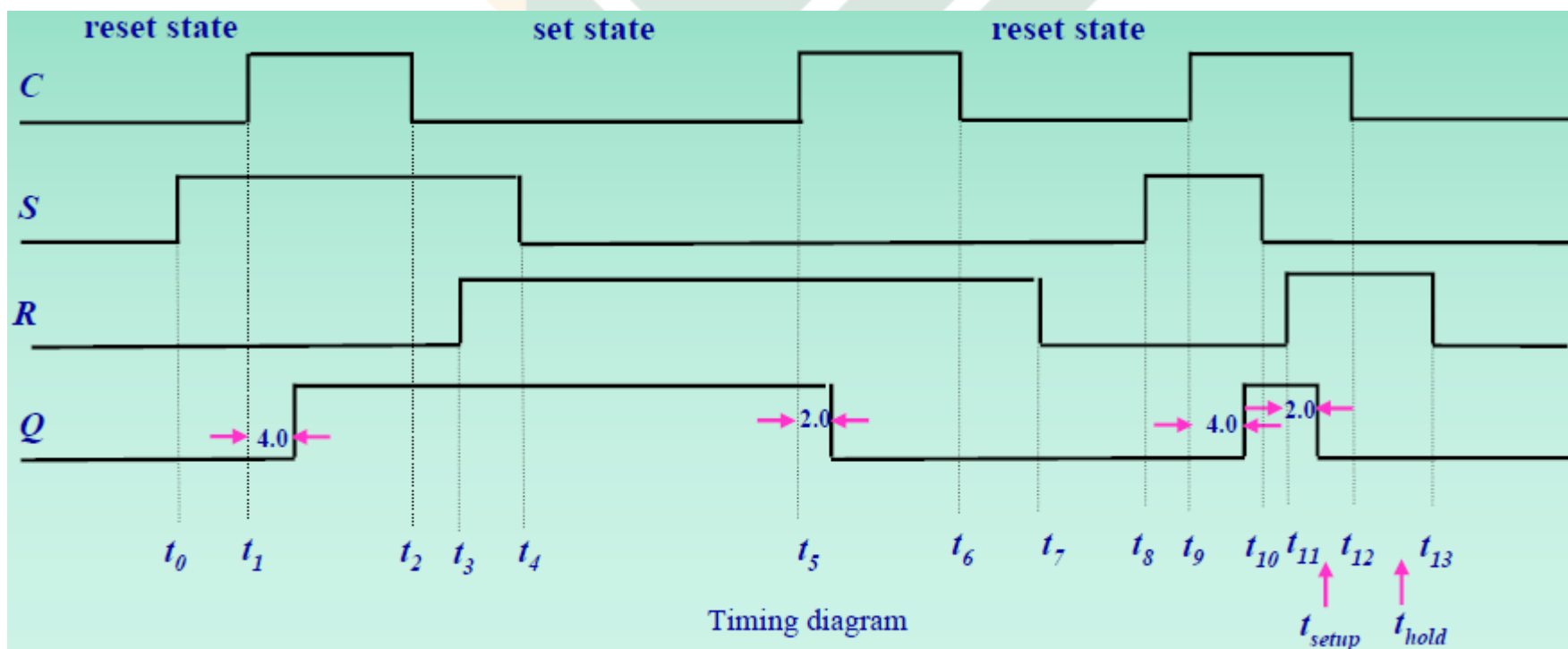
Gated SR latch

Control signal C activates the latch.

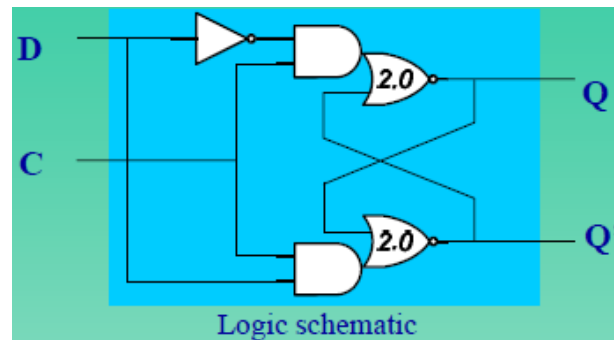
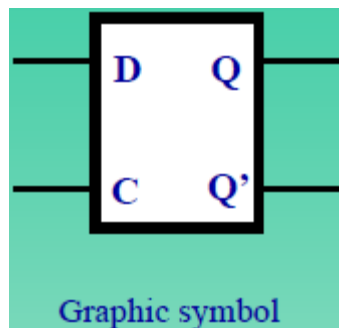


C	S	R	Q	Q(next)	
0	X	X	0	0	(inactive)
0	X	X	1	1	(inactive)
1	0	0	0	0	(hold)
1	0	0	1	1	(hold)
1	0	1	X	0	(reset)
1	1	0	X	1	(set)
1	1	1	X	NA	(?)

Truth table

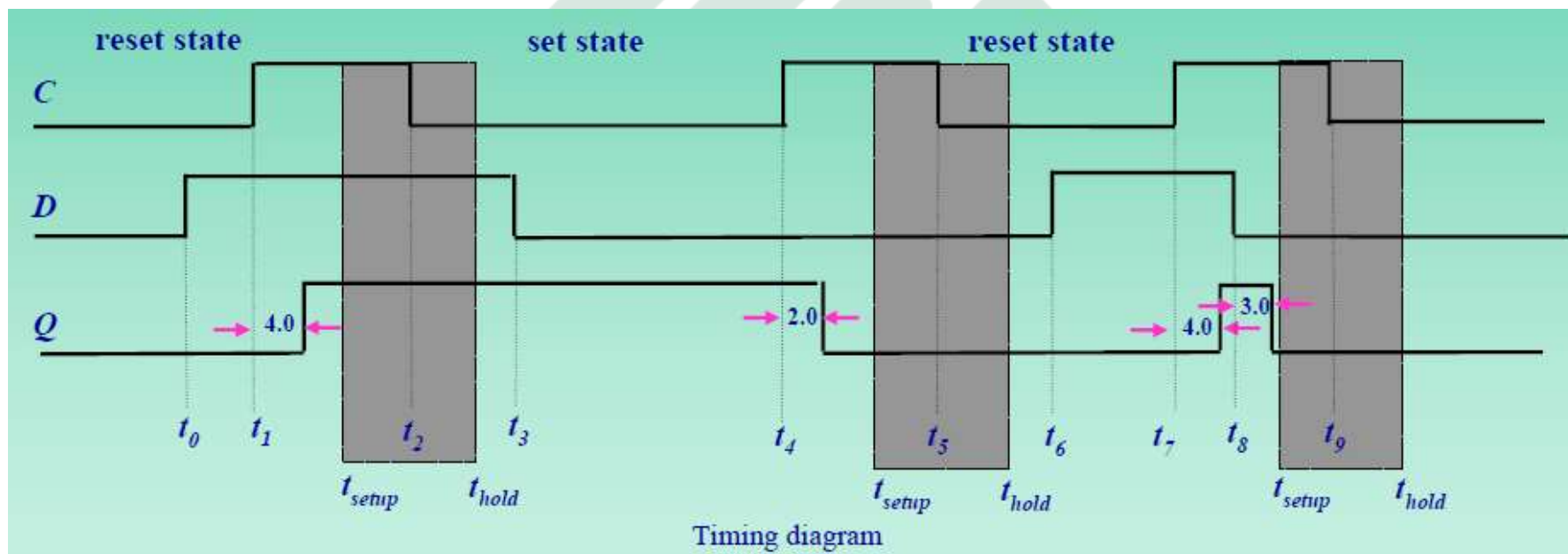


Gated D latch (Control signal C activates the latch)



C	D	Q	Q(next)
0	X	0	0
0	X	1	1
1	0	X	0
1	1	X	1

Truth table



Setup time is minimum time inputs must be stable before C go falling

Hold time is minimum time inputs must be stable after C go falling

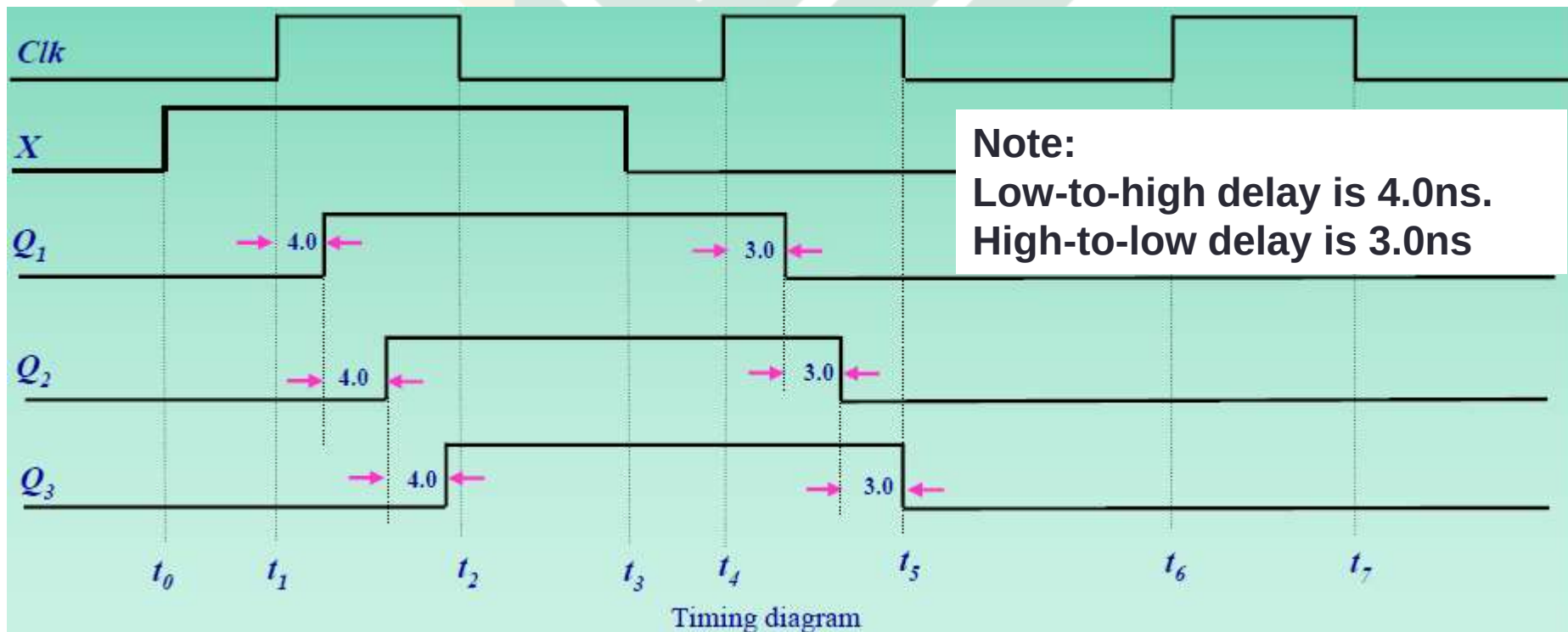
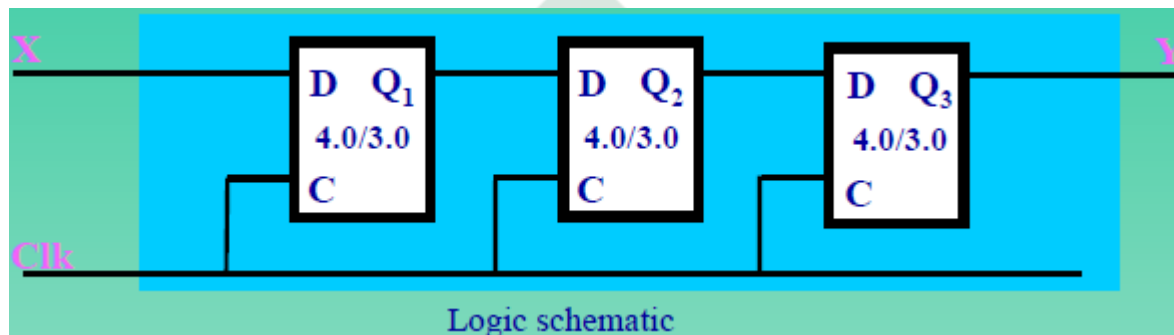
Q follows D while C is asserted as long as D satisfies setup and hold time restrictions

Flip-Flops (FF)

- **Latches** are level-sensitive since they respond to input changes *during clock width* → It is difficult to work.
- **Flip-Flops** respond to input changes only *during the change in clock signal* **edge**.
- **FFs** are easy to work with though more expensive than latches.
- Two styles of flip-flops are available.
 1. **master-slave (MS)**
 2. **edge-triggered (ET)**

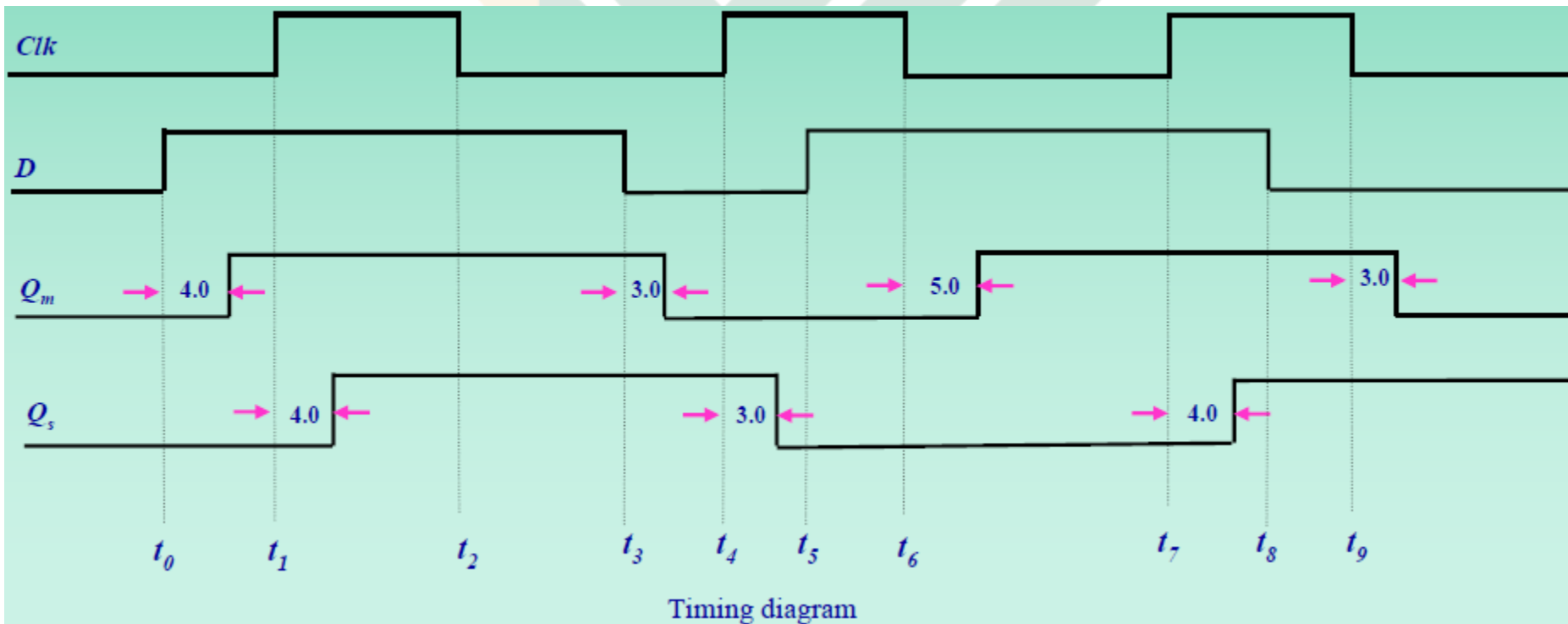
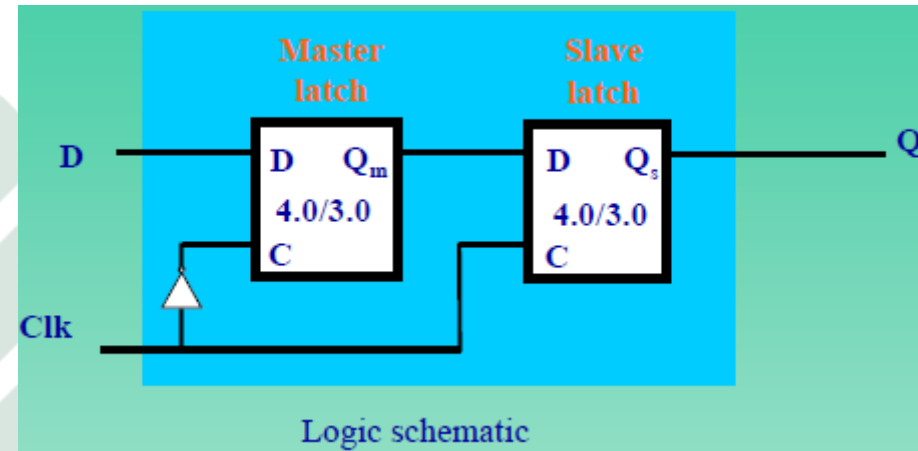
Erroneous shifting with D-latches

Erroneous operation is possible with level-sensitive latches

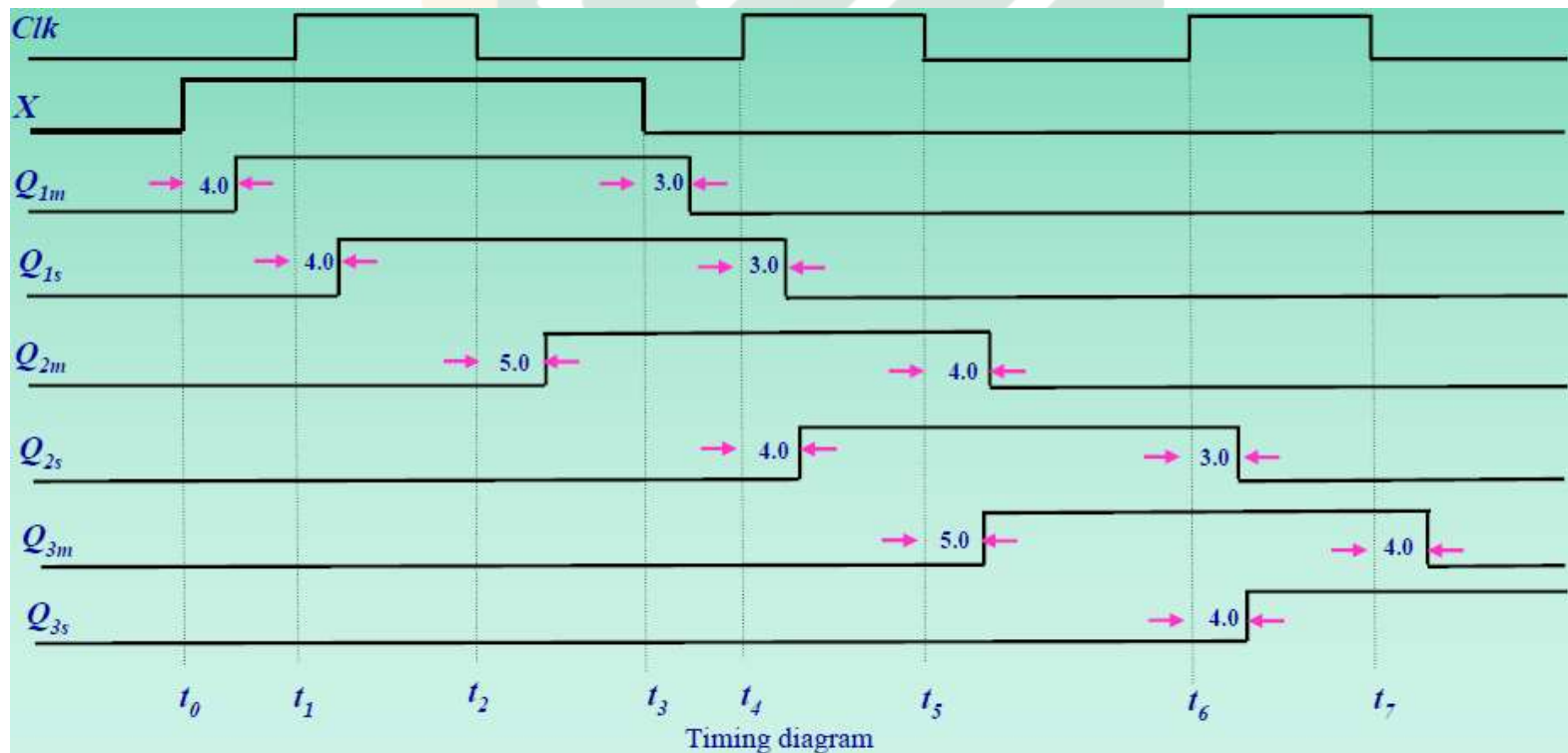
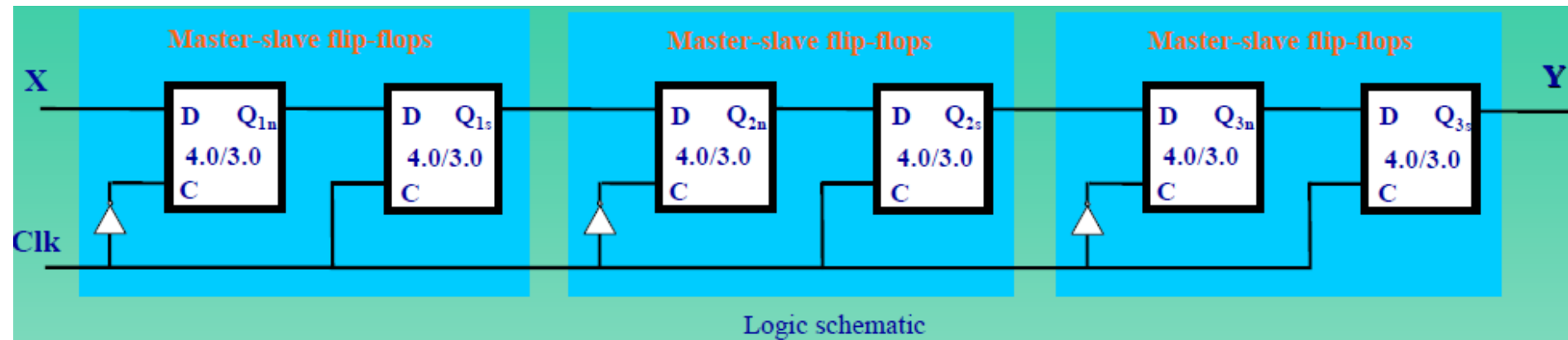


Master-slave D-FF

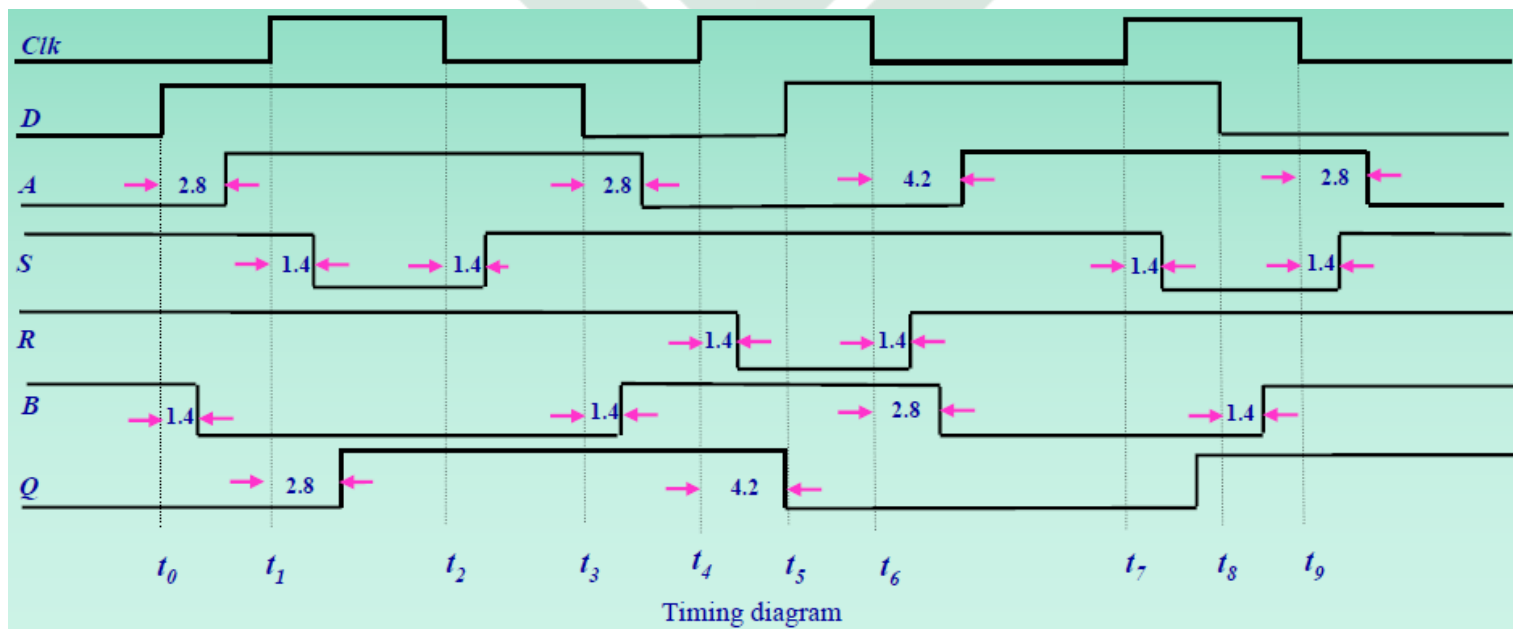
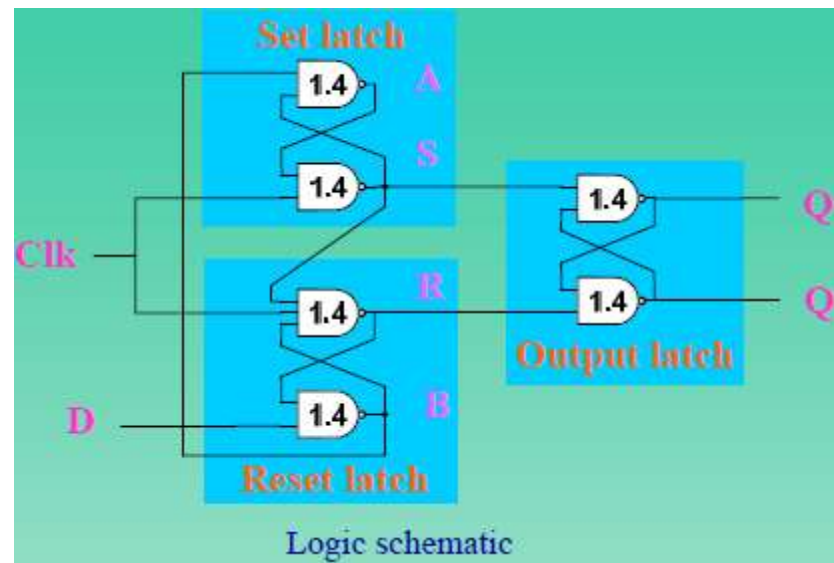
In a MS flip-flop, D is sampled and stored at the *rising edge* of the Clk signal



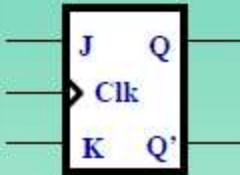
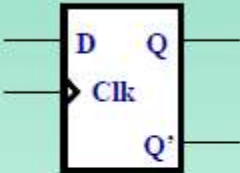
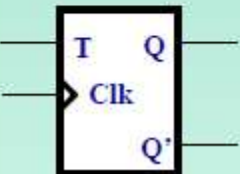
Shifting with Master-Slave D-FF



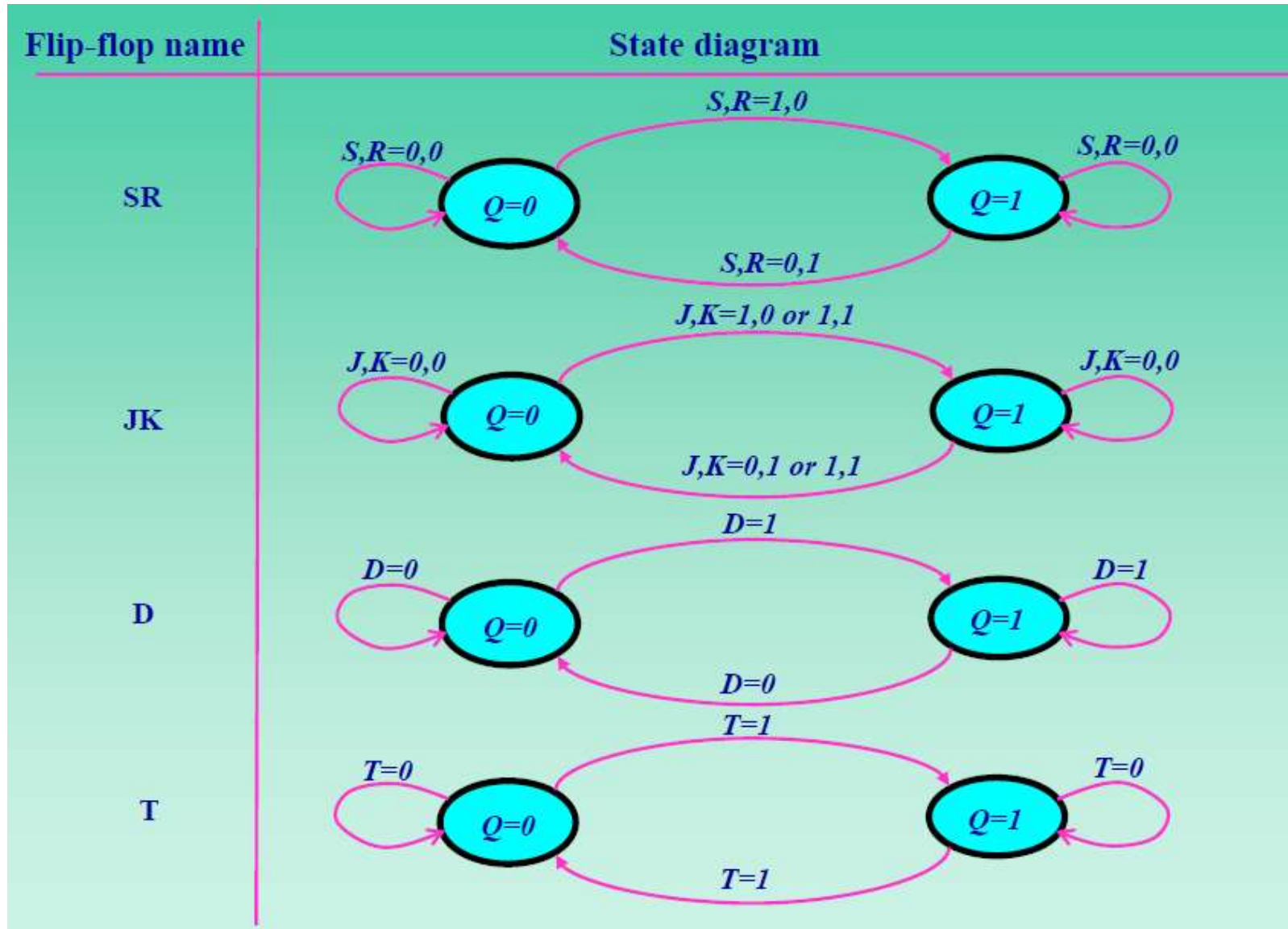
Positive Edge-triggered D-FF



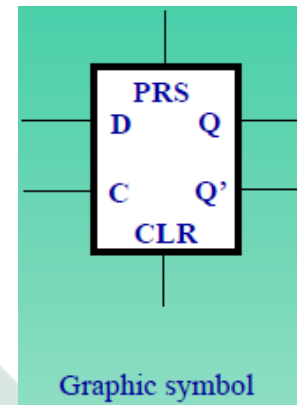
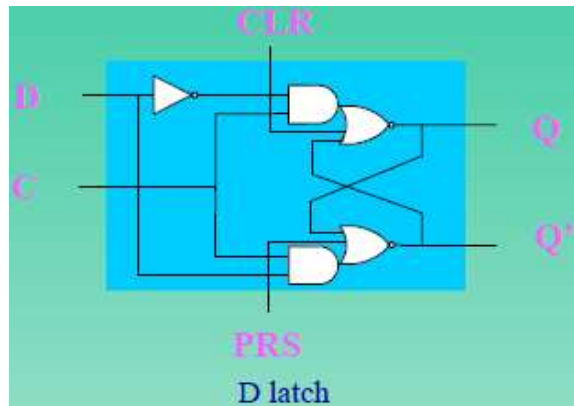
Flip-flop types

Flip-flop name	Flip-flop symbol	Characteristic table	Characteristic equation	Excitation table																																			
SR		<table><tr><th>S</th><th>R</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>NA</td></tr></table>	S	R	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q(next)=S+R'Q$ $SR=0$	<table><tr><th>Q</th><th>Q(next)</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q(next)	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table><tr><th>J</th><th>K</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Q'</td></tr></table>	J	K	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q(next)=JQ'+K'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>X</td></tr><tr><td>1</td><td>0</td><td>X</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q(next)	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				
D		<table><tr><th>D</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q(next)	0	0	1	1	$Q(next)=D$	<table><tr><th>Q</th><th>Q(next)</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q(next)	D	0	0	0	0	1	1	1	0	0	1	1	1														
D	Q(next)																																						
0	0																																						
1	1																																						
Q	Q(next)	D																																					
0	0	0																																					
0	1	1																																					
1	0	0																																					
1	1	1																																					
T		<table><tr><th>T</th><th>Q(next)</th></tr><tr><td>0</td><td>Q</td></tr><tr><td>1</td><td>Q'</td></tr></table>	T	Q(next)	0	Q	1	Q'	$Q(next)=TQ'+T'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q(next)	T	0	0	0	0	1	1	1	0	1	1	1	0														
T	Q(next)																																						
0	Q																																						
1	Q'																																						
Q	Q(next)	T																																					
0	0	0																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					

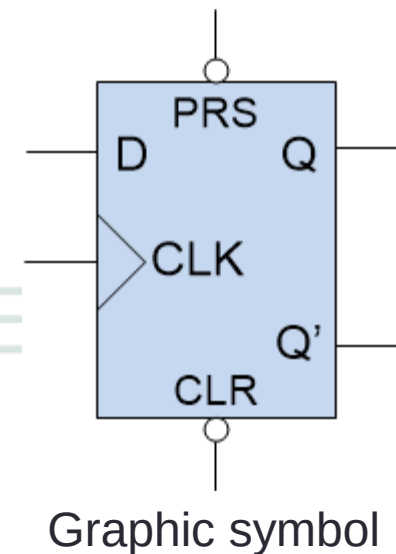
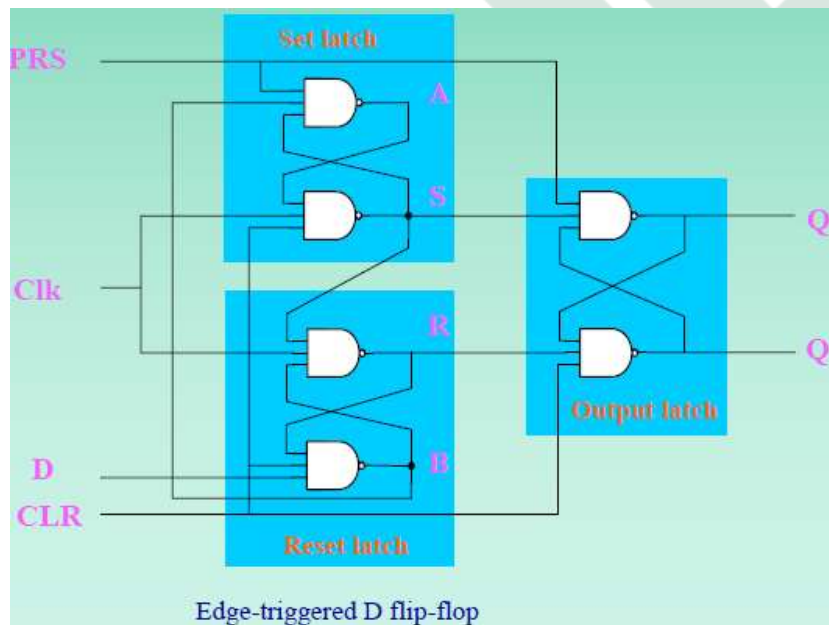
State diagrams for different Flip-flops



A D-Latch asynchronous inputs

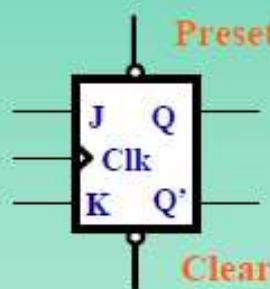
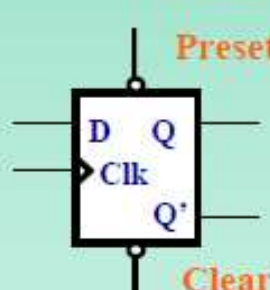
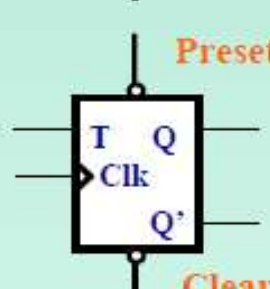


A D-FF with asynchronous inputs



Graphic symbols for FFs with asynchronous inputs

COMPUTER

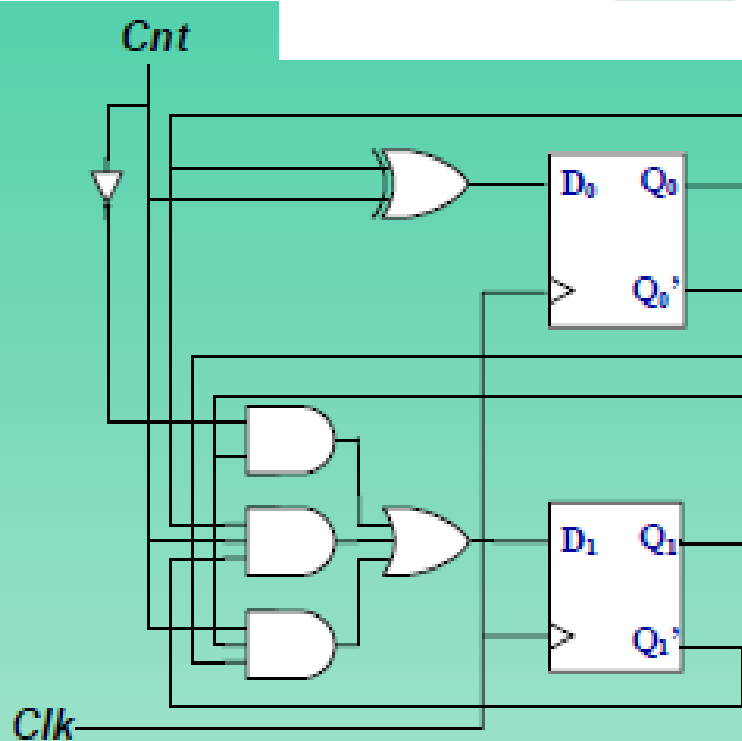
Flip-flop name	Flip-flop symbol
SR	 The SR flip-flop symbol is a square block with a clock input (Clk) on the left. It has two data inputs, S and R, on the left. It has two data outputs, Q and Q', on the right. It has two asynchronous inputs: Preset at the top and Clear at the bottom, both indicated by lines with small circles at the connection points.
JK	 The JK flip-flop symbol is a square block with a clock input (Clk) on the left. It has two data inputs, J and K, on the left. It has two data outputs, Q and Q', on the right. It has two asynchronous inputs: Preset at the top and Clear at the bottom, both indicated by lines with small circles at the connection points.
D	 The D flip-flop symbol is a square block with a clock input (Clk) on the left. It has one data input, D, on the left. It has two data outputs, Q and Q', on the right. It has two asynchronous inputs: Preset at the top and Clear at the bottom, both indicated by lines with small circles at the connection points.
T	 The T flip-flop symbol is a square block with a clock input (Clk) on the left. It has one data input, T, on the left. It has two data outputs, Q and Q', on the right. It has two asynchronous inputs: Preset at the top and Clear at the bottom, both indicated by lines with small circles at the connection points.

Analysis of sequential logic

- **Excitation equations** are Boolean expressions of the FF's inputs.
- **Next-state equations** are Boolean expressions representing the next value of the FF's outputs (Q).
- **Next-state table** (similar to next-state equations) gives the next value of FF's outputs for each input value and state of FFs.
- **Analysis of a sequential circuit** is a procedure that produces the next-state table, state diagram and timing diagram from the logic schematic of the circuit.
- The analysis gives the answer to the following questions:
 1. What is the next state?
 2. What is the output?
 3. What is the function of the circuit?

Analysis of a sequential circuit

- **Example:** Modulo-4 counter
- **Problem:** Derive the state table and state diagram for the sequential circuit represented by the schematic below.



Logic schematic

Step 1

$$D_0 = Cnt \oplus Q_0 = Cnt' Q_0 + Cnt Q_0'$$

$$D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Excitation equation

$$Q_0(next) = D_0 = Cnt' Q_0 + Cnt Q_0'$$

$$Q_1(next) = D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Next-state equation

Note: D-FF $\rightarrow Q_{next} = D$

Analysis of a sequential circuit

- **Example:** Modulo-4 counter
- **Problem:** Derive the state table and state diagram for the sequential circuit represented by the schematic below.

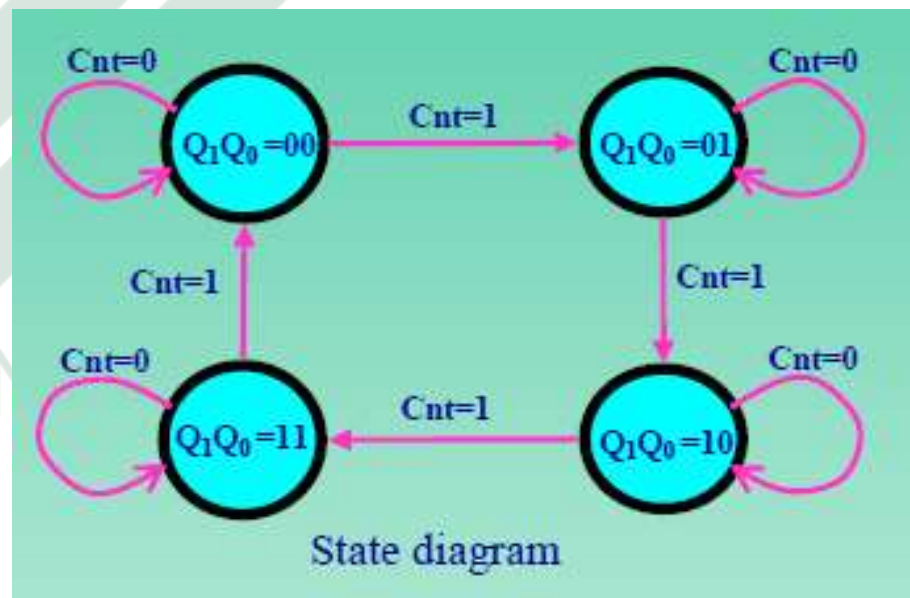
Step 2

Next-state equation

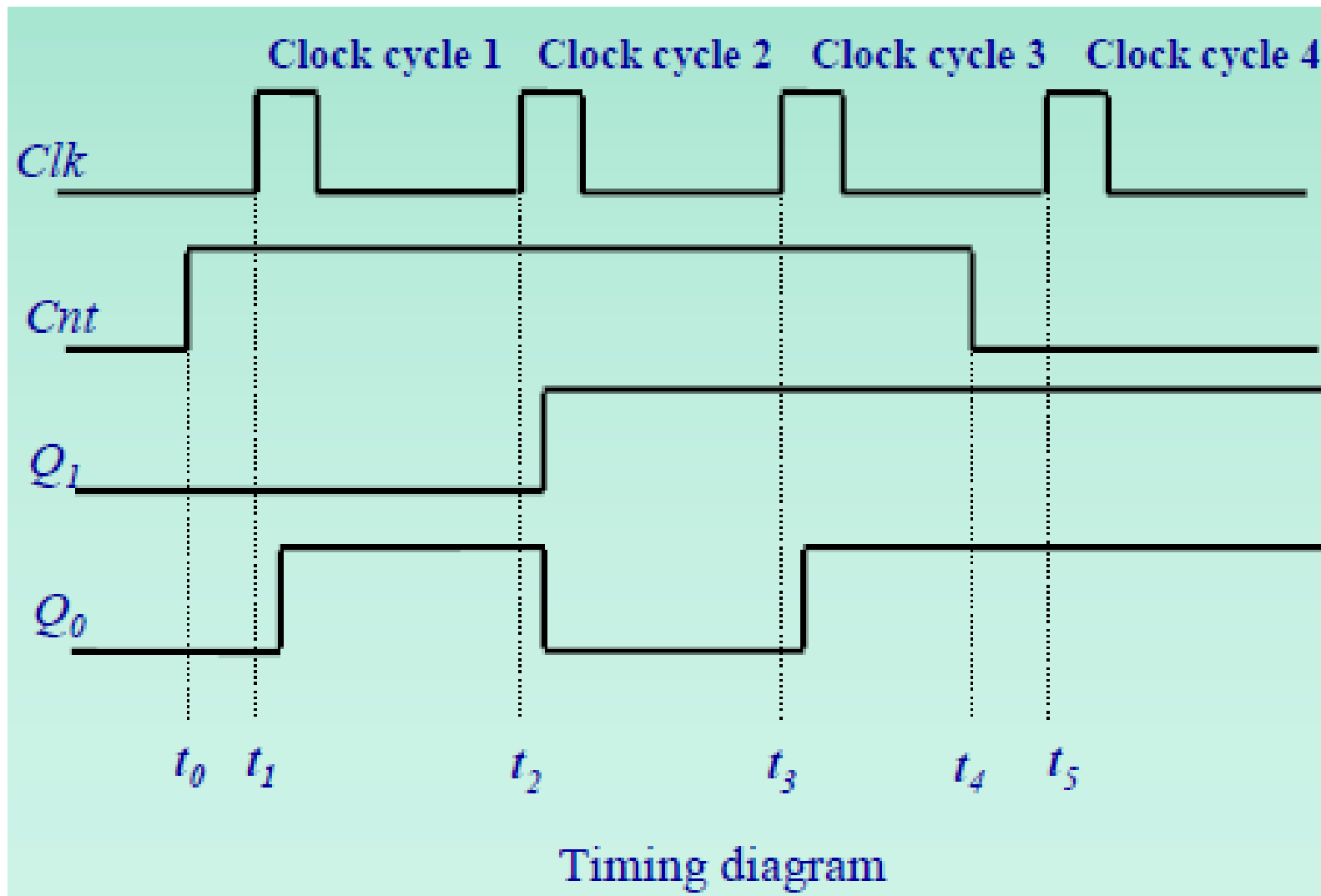
<i>PRESENT STATE</i> Q_1Q_0	<i>NEXT STATE</i> $Q_1(next) Q_0(next)$	
	Cnt=0	Cnt=1
0 0	0 0	0 1
0 1	0 1	1 0
1 0	1 0	1 1
1 1	1 1	0 0

State table

Step 3



Analysis of a sequential circuit

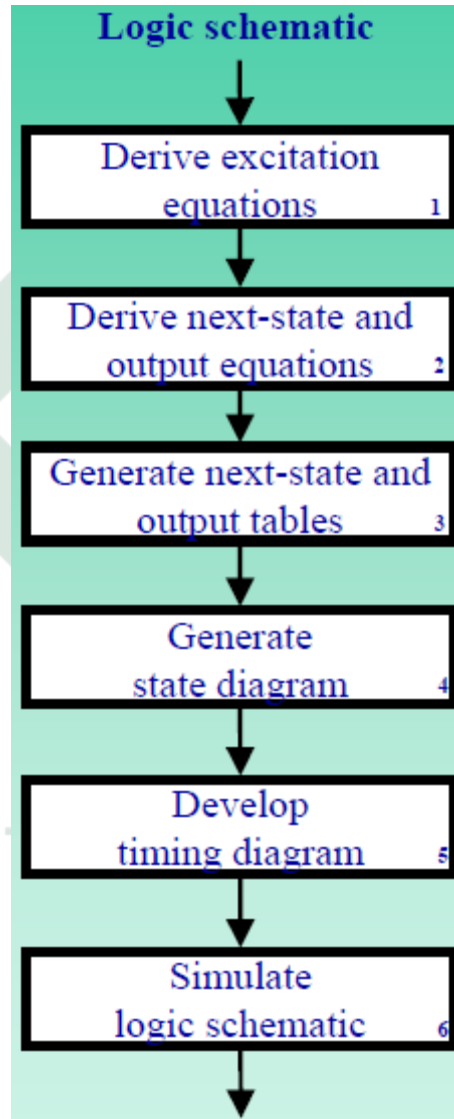


Bài tập

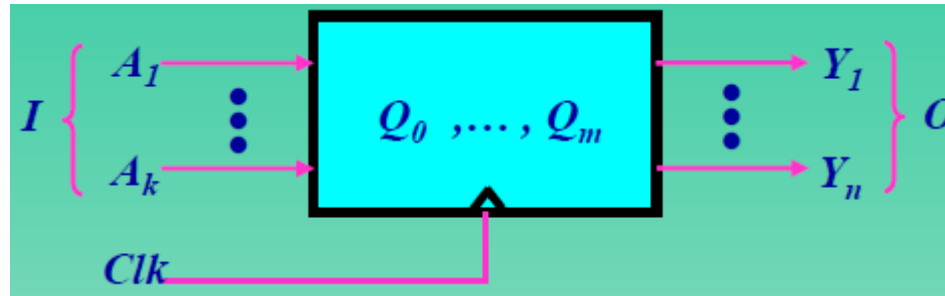
- Tính giá trị frequency và duty-cycle. Khi biết các giá trị của độ rộng xung clock (clock's width) và chu kì xung clock (clock's period):
 - 5ns & 20ns
 - 10ns & 100ns
- Dùng D flip-flop thiết kế cho mạch đếm tuần tự có chu trình đếm như sau: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, ...
- Dùng T flip-flop thiết kế cho mạch đếm tuần tự có chu trình đếm như sau: 0, 1, 3, 6, 7, 0, ...

Note: Xem lại phần thiết kế bộ đếm tuần tự trong môn Nhập môn Mạch số

Analysis procedure for sequential circuits



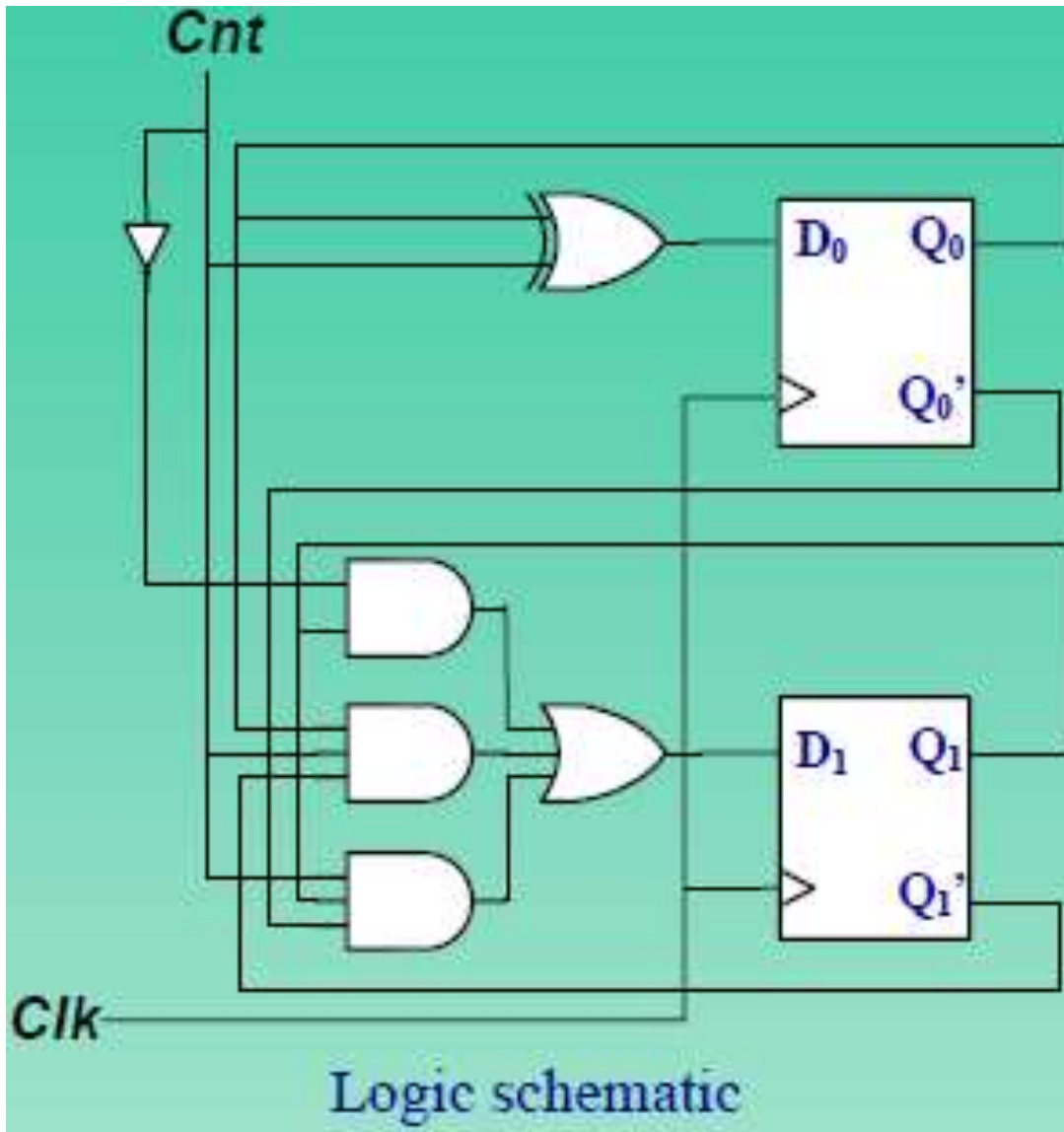
Finite State Machine model



- The Finite State Machine (FSM) can be defined abstractly as the quintuple (a set of five things)

$$\langle S, I, O, f, h \rangle$$
- where **S**, **I**, and **O** represent a set of *states*, set of *inputs* and a set of *outputs*, respectively; and **f** and **h** represent the *next-state* and the *output functions*.
- $f : S \times I \rightarrow S$
- $h : S \times I \rightarrow O$ (Mealy-type)
 $S \rightarrow O$ (Moore-type)
- $S = Q_1 \times Q_2 \times \dots \times Q_m$
- $I = A_1 \times A_2 \times \dots \times A_k$
- $O = Y_1 \times Y_2 \times \dots \times Y_n$

FSM model of modulo-4 counter (Moore)



$$D_0 = Cnt \oplus Q_0 = Cnt' Q_0 + Cnt Q_0'$$

$$D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Excitation equation

$$Q_0(next) = D_0 = Cnt' Q_0 + Cnt Q_0'$$

$$Q_1(next) = D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Next-state equation

FSM model of modulo-4 counter (Moore)

$$D_0 = Cnt \oplus Q_0 = Cnt' Q_0 + Cnt Q_0'$$

$$D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Excitation equation

$$Q_0(next) = D_0 = Cnt' Q_0 + Cnt Q_0'$$

$$Q_1(next) = D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Next-state equation

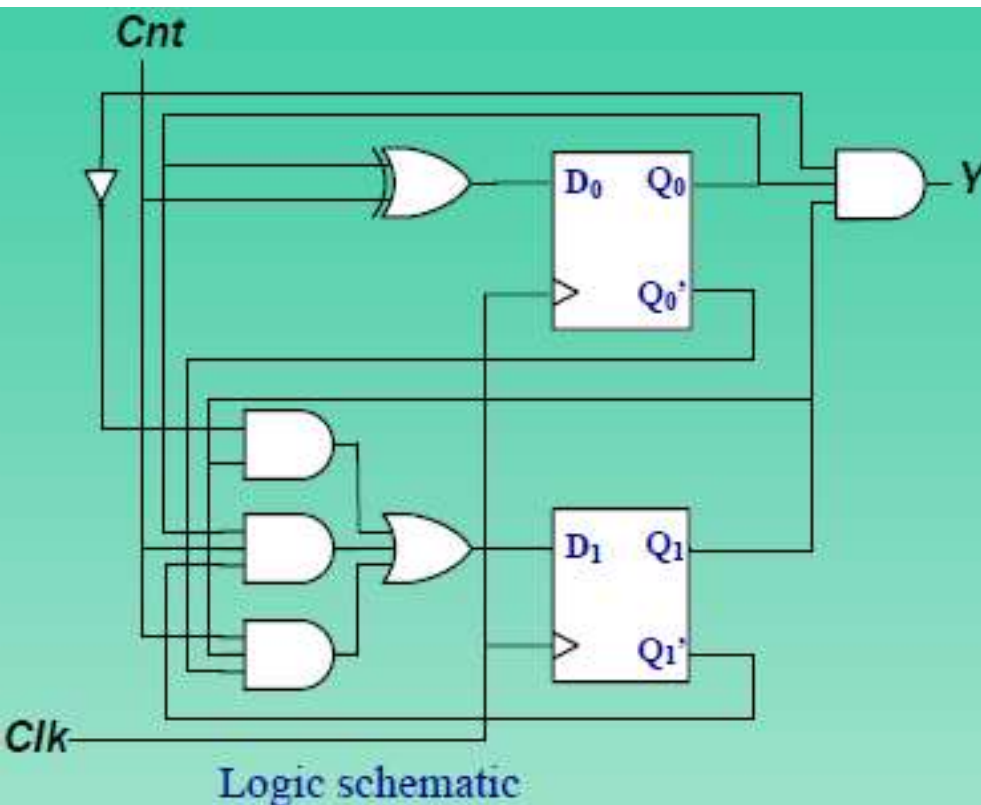
PRESENT STATE $Q_1 Q_0$	NEXT STATE $Q_1(next) Q_0(next)$	
	Cnt=0	Cnt=1
0 0	0 0	0 1
0 1	0 1	1 0
1 0	1 0	1 1
1 1	1 1	0 0

State table

PRESENT STATE	NEXT STATE ($S \times I \rightarrow S$)		OUTPUTS ($S \rightarrow O$)
	i_0	i_1	
S_0	S_0	S_1	O_0
S_1	S_1	S_2	O_0
S_2	S_2	S_3	O_0
S_3	S_3	S_0	O_1

FSM model of modulo-4 counter

FSM model of modulo 4 counter (Mealy)



$$D_0 = Cnt \oplus Q_0 = Cnt' Q_0 + Cnt Q_0'$$

$$D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

Excitation equation

$$Q_0(\text{next}) = D_0 = Cnt' Q_0 + Cnt Q_0'$$

$$Q_1(\text{next}) = D_1 = Cnt' Q_1 + Cnt Q_1' Q_0 + Cnt Q_1 Q_0'$$

$$Y = Cnt Q_0 Q_1$$

Next-state and output equations

FSM model of modulo 4 counter (Mealy)

<i>PRESENT STATE</i> Q_1Q_0	<i>NEXT STATE /OUTPUTS</i> $Q_1(next) Q_0(next)/Y$	
	Cnt=0	Cnt=1
0 0	0 0 / 0	0 1 / 0
0 1	0 1 / 0	1 0 / 0
1 0	1 0 / 0	1 1 / 0
1 1	1 1 / 0	0 0 / 1

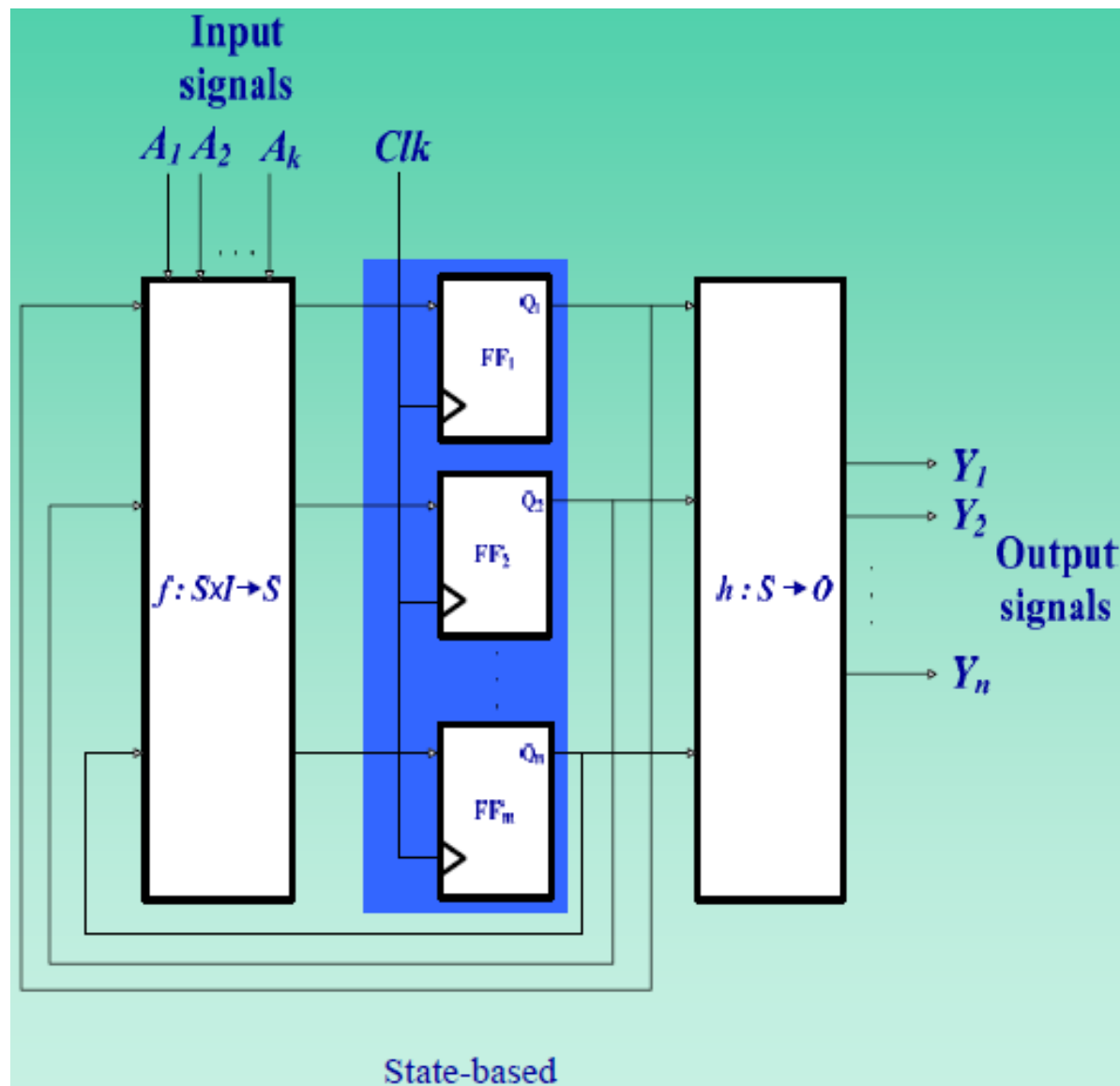
State and output table

PRESENT STATE	NEXT STATE($SxI \rightarrow S$)/ OUTPUT($SxH \rightarrow O$)	
	i_0	i_1
S_0	S_0/O_0	S_1/O_0
S_1	S_1/O_0	S_2/O_0
S_2	S_2/O_0	S_3/O_0
S_3	S_3/O_0	S_0/O_1

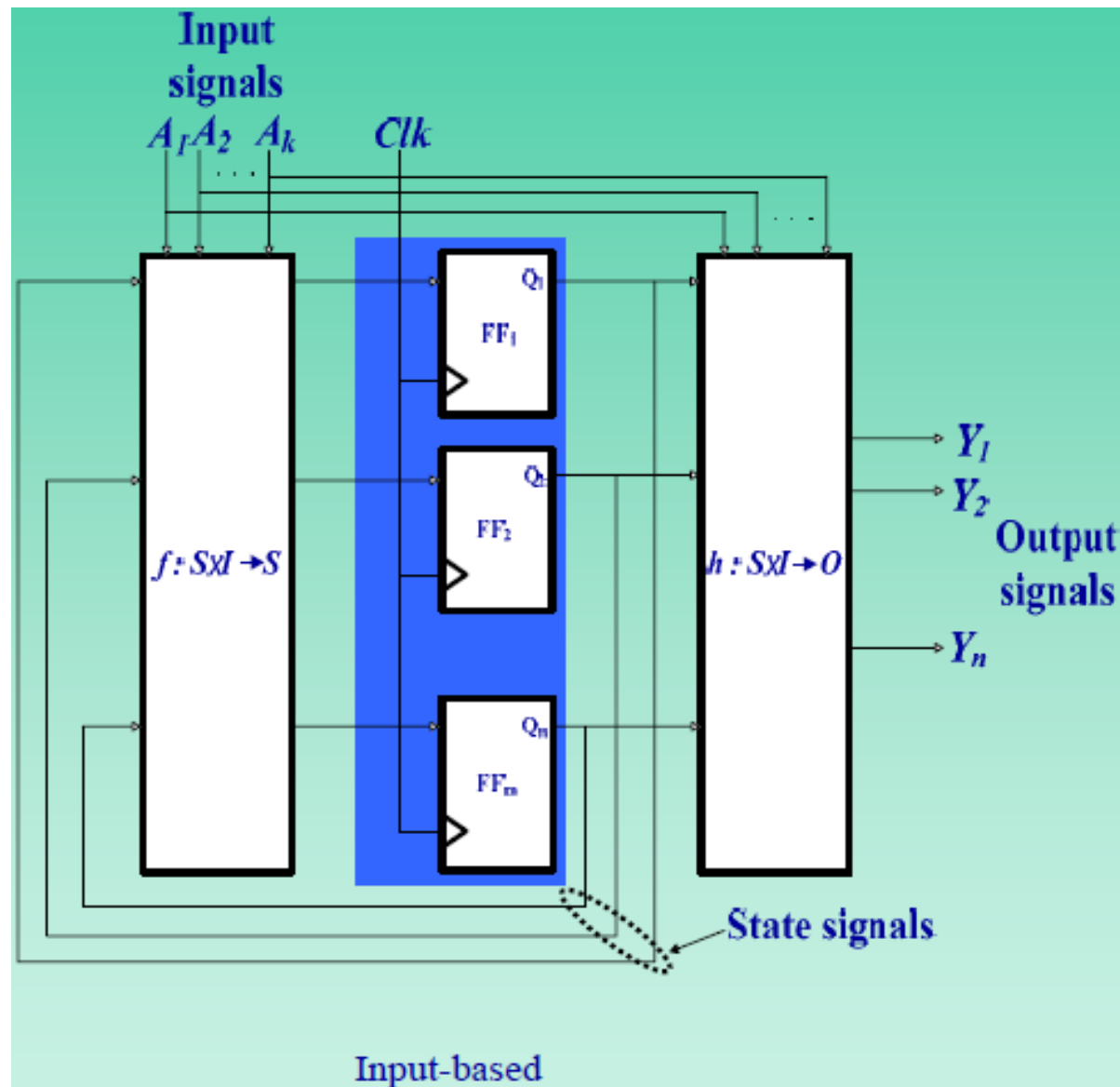
FSM model of modulo-4 counter

Note: Output is dependent on the present state and the input of the FSM.

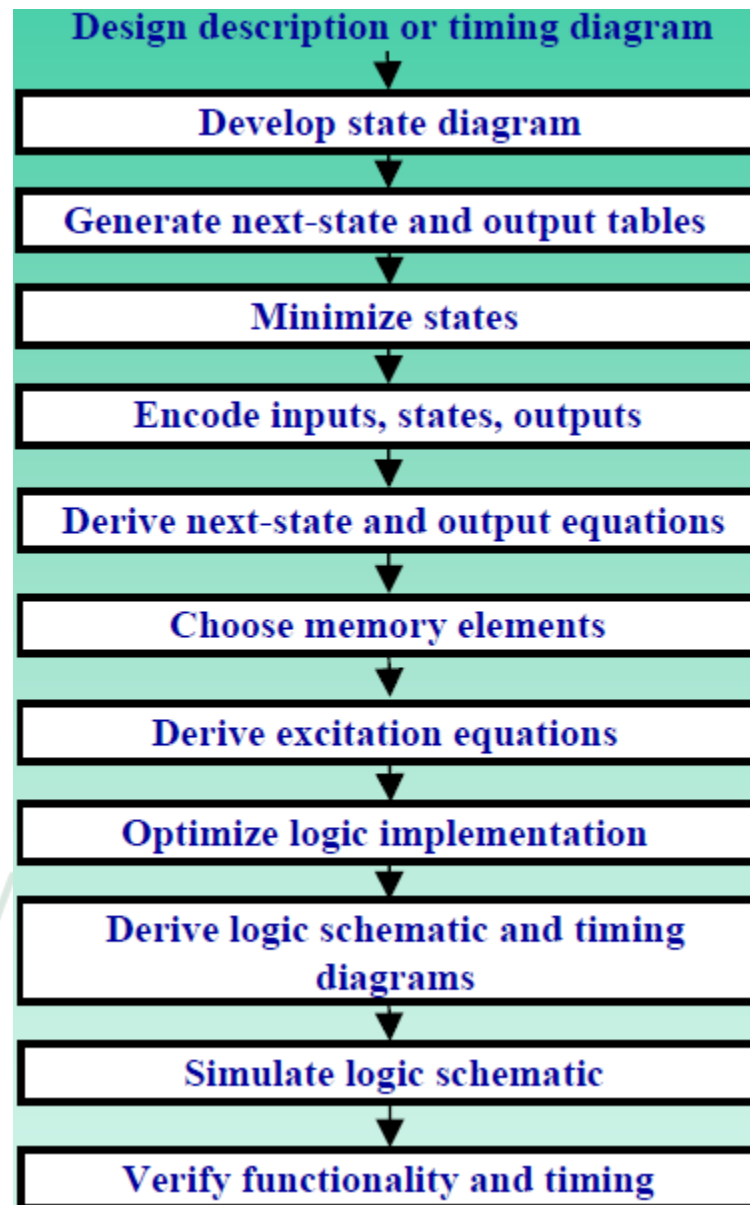
FSM implementations for Moore model



FSM implementations for Mealy model



Synthesis procedure for sequential logic



State diagram for a modulo 3 up/down counter

➤ **Example:** Modulo-3 up-down counter

➤ **Problem:**

Derive the *state diagram* for an up-down, modulo-3 counter. The counter has two inputs: count enable (C) and count direction (D).

When $C = 1$, the counter will count in the direction specified by D, and it will stop counting when $C = 0$.

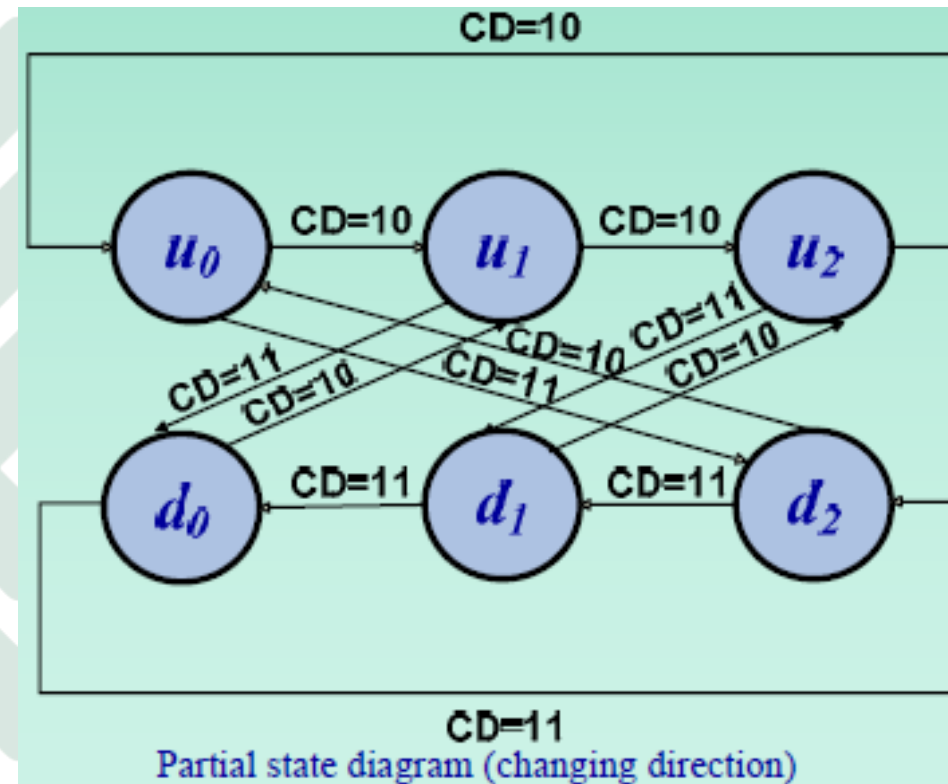
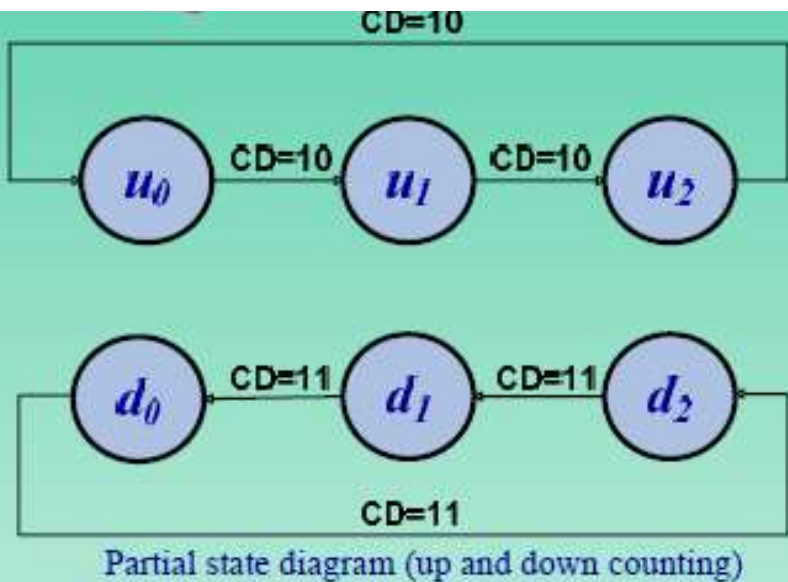
The counter will count up when $D=0$ and down when $D=1$.

The counter has one output Y which *will be asserted* when the counter:

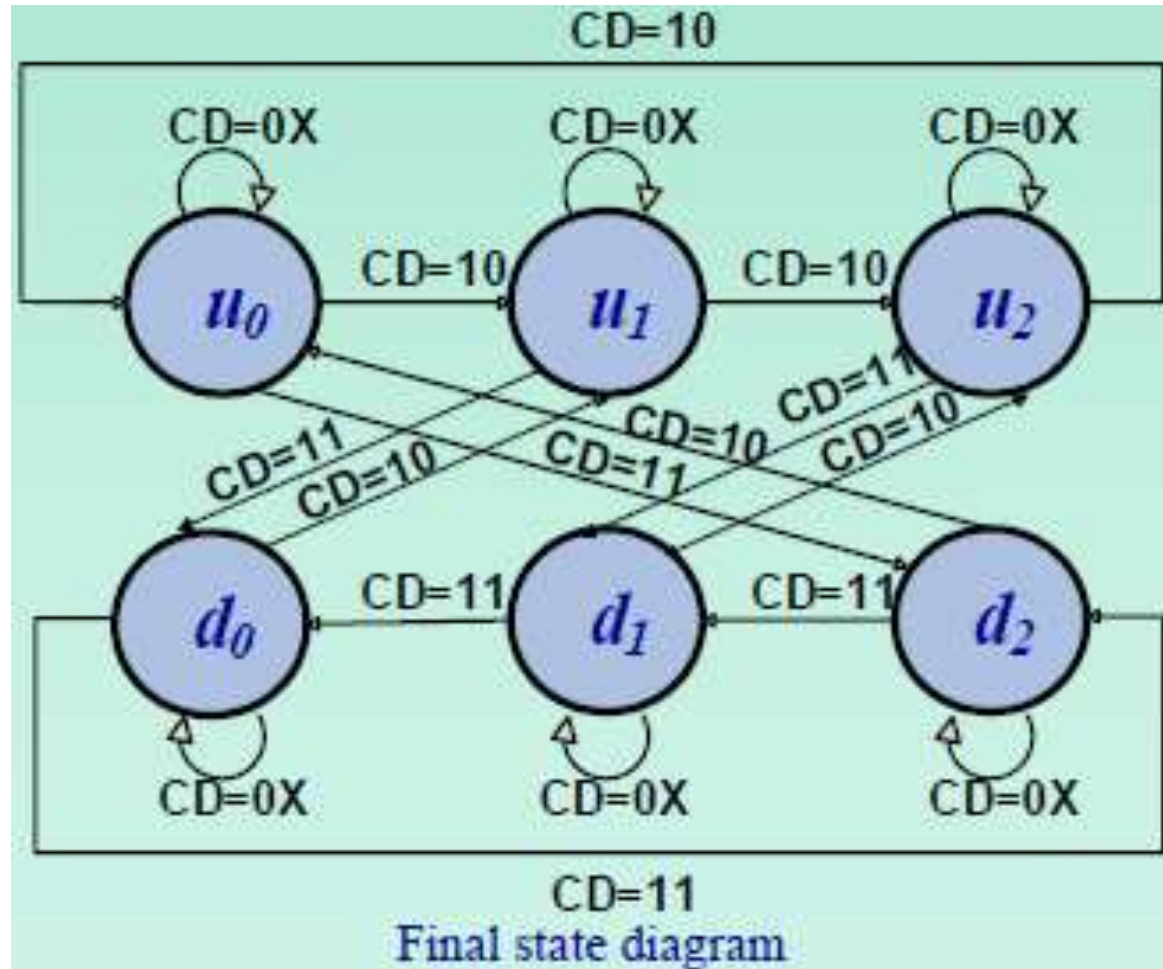
- reaches 2 while counting up ~~down~~
- or - reaches 0 while counting down ~~up~~.



State diagram for a modulo 3 up/down counter



State diagram for a modulo 3 up/down counter (cont.)



State minimization

- State minimization reduces the number of states, and therefore, number of flip-flops needed to implement the circuit also reduce.
- State minimization is based on the concept of behavioral equivalence in which two states are equivalent if they produce the same sequence of next-state and output symbols for every sequence of input symbols.

More formally, two states, s_j and s_k in a FSM are said to be equivalent if the following two conditions are true

- Condition 1: Both states s_j and s_k produce the same output symbol for every input symbol i : that is, $h(s_j, i) = h(s_k, i)$
- Condition 2: Both states have equivalent next states for every input symbol i : that is, $f(s_j, i) = f(s_k, i)$

- Minimization procedure:

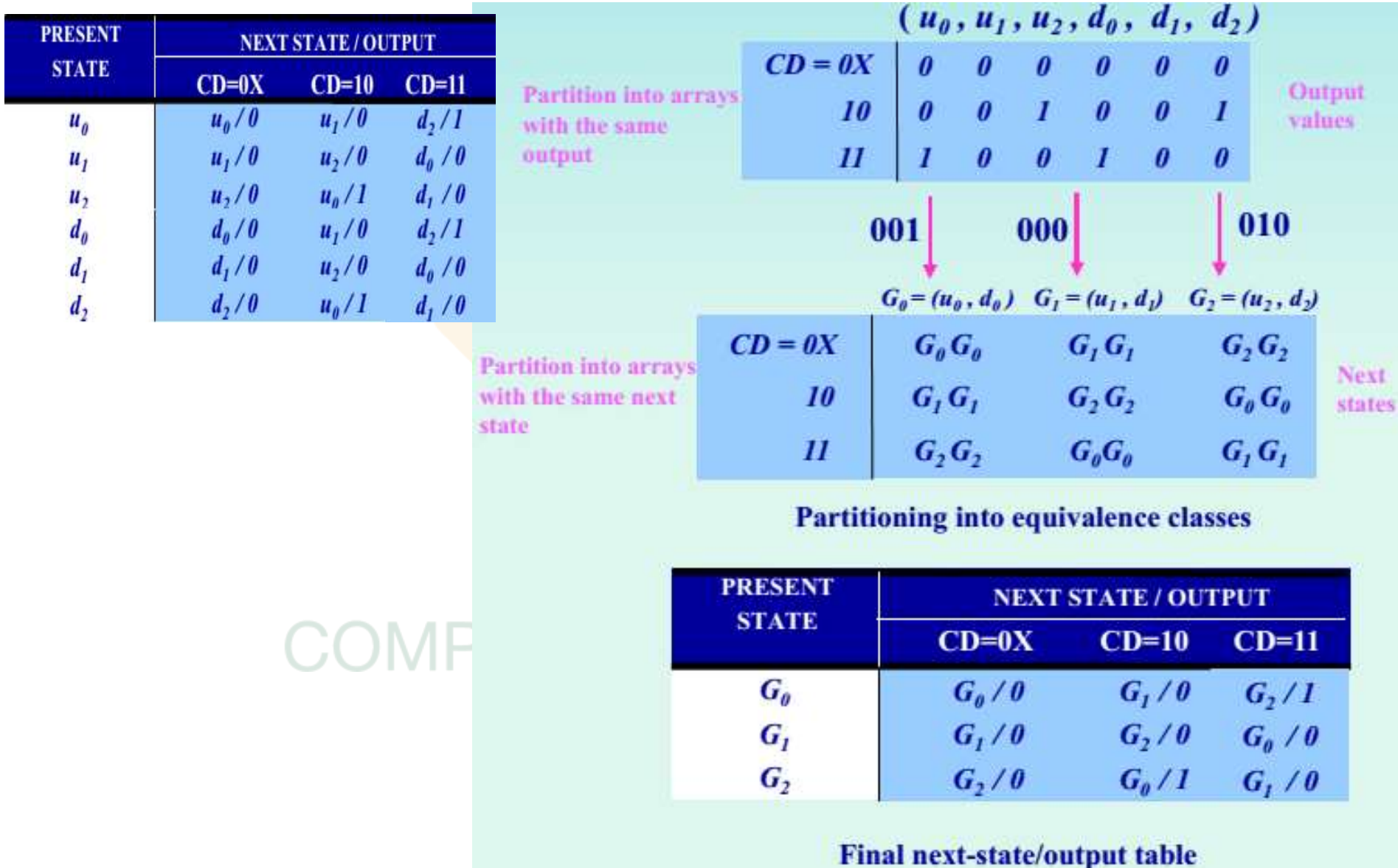
1. Partition states into equivalence classes
2. Construct new FSM with one state for each equivalence class

State reduction for modulo-3 counter

- Example: State reduction
- Problem: Derive the minimal-state FSM for the modulo-3 counter.

PRESENT STATE	NEXT STATE / OUTPUT		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_1 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_1 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$

State reduction for modulo-3 counter



State reduction with implication table

Implication table

- Step 1: Enter x for any pair of states that differ in output values
- Step2: Enter implied equivalent states for every pair of states
- Step3: Enter x for the non equivalent next-state pair
- Step4: Form equivalence classes using transitivity : $s_1 \equiv s_2 \text{ \& } s_2 \equiv s_3 \Rightarrow s_1 \equiv s_3$

s_1						
s_2						
s_3						
s_4						
s_5						
s_6						
	s_0	s_1	s_2	s_3	s_4	s_5

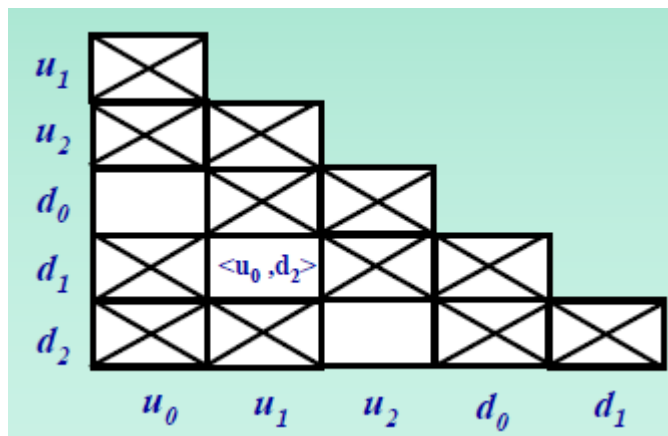
Implication table

COMPUTER ENGINEERING

State reduction with implication table

- Example: State reductions with implication table.
- Problem: Find the minimal number of states for the FSM specified by the table below.

PRESENT STATE	NEXT STATE		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_0 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_2 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$



Equivalence classes:

$\langle u_0, d_0 \rangle$

$\langle u_1 \rangle$

$\langle d_1 \rangle$

$\langle u_2, d_2 \rangle$

Implication table
for the table above



COMPUTER ENGINEERING

State encoding

ENCODING NUMBER	s_0	s_1	s_2	s_3
1	00	01	10	11
2	00	01	11	10
3	00	10	01	11
4	00	10	11	01
5	00	11	01	10
6	00	11	10	01
7	01	00	10	11
8	01	00	11	10
9	01	10	00	11
10	01	10	11	00
11	01	11	00	10
12	01	11	10	00
13	10	00	01	11
14	10	00	11	01
15	10	01	00	11
16	10	01	11	00
17	10	11	00	01
18	10	11	01	00
19	11	00	01	10
20	11	00	10	01
21	11	01	00	10
22	11	01	10	00
23	11	10	00	01
24	11	10	01	00

24 encodings of four states

The cost and delay of FSM implementation depends on encoding of symbolic states

For example, four states can be encoded in $4!=24$ different ways.

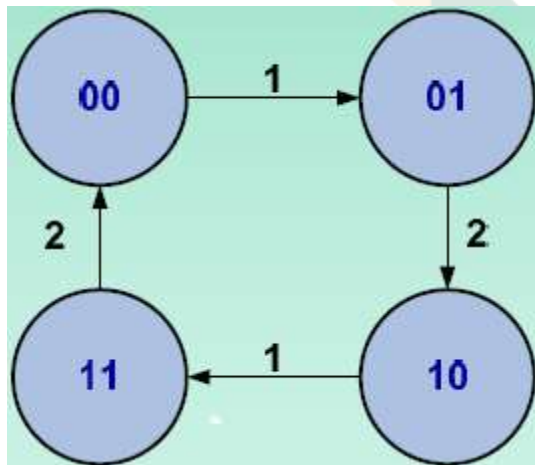
There are more than $n!$ different encodings for n states. Exploration of all encodings is impossible. Thus, we use heuristics.

Three different heuristics:

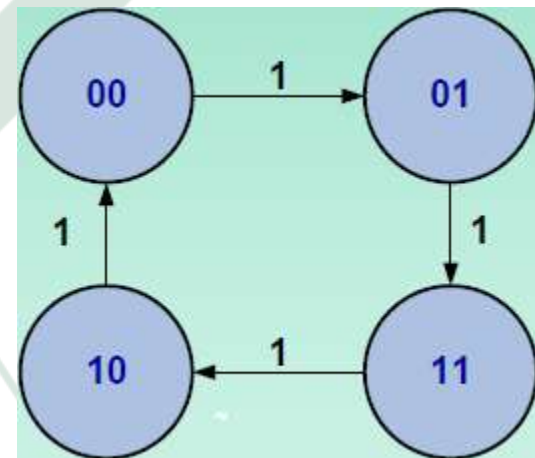
- minimum-bit-change
- prioritized adjacency
- one-hot encoding

Minimum-bit-change

- Minimum-bit change strategy assigns codes to states so that the total number of bit changes for *all state transitions* is *minimized*.
- Example: Two different encodings for 2-bit binary counter



Straightforward encoding
(total weight is 6)



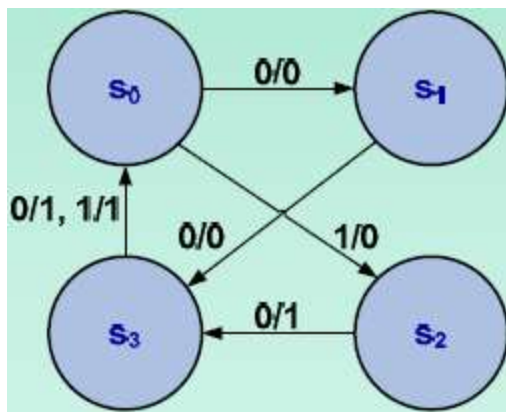
Minimum-bit-change encoding
(total weight is 4)

Prioritized adjacency

Prioritized adjacency strategy assigns adjacent encodings to all states with common *source*, common *destination* or common *output*.

- The **highest priority** is given to states that have the same next state for a given input value → the same next-state encoding will appear in adjacent entries in the Karnaugh map.
- The **second priority** is given to the next states of the same state → the next state also may appear adjacent in the Karnaugh map.
- The **third priority** is given to states that have the same output value for the same input values → they may be adjacent in the *output map*.

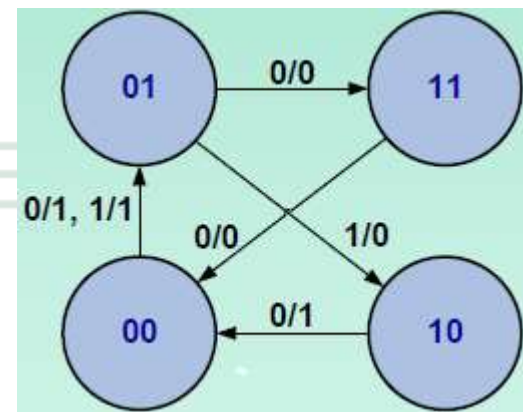
Example: encoding based on prioritized adjacency.



Initial state diagram

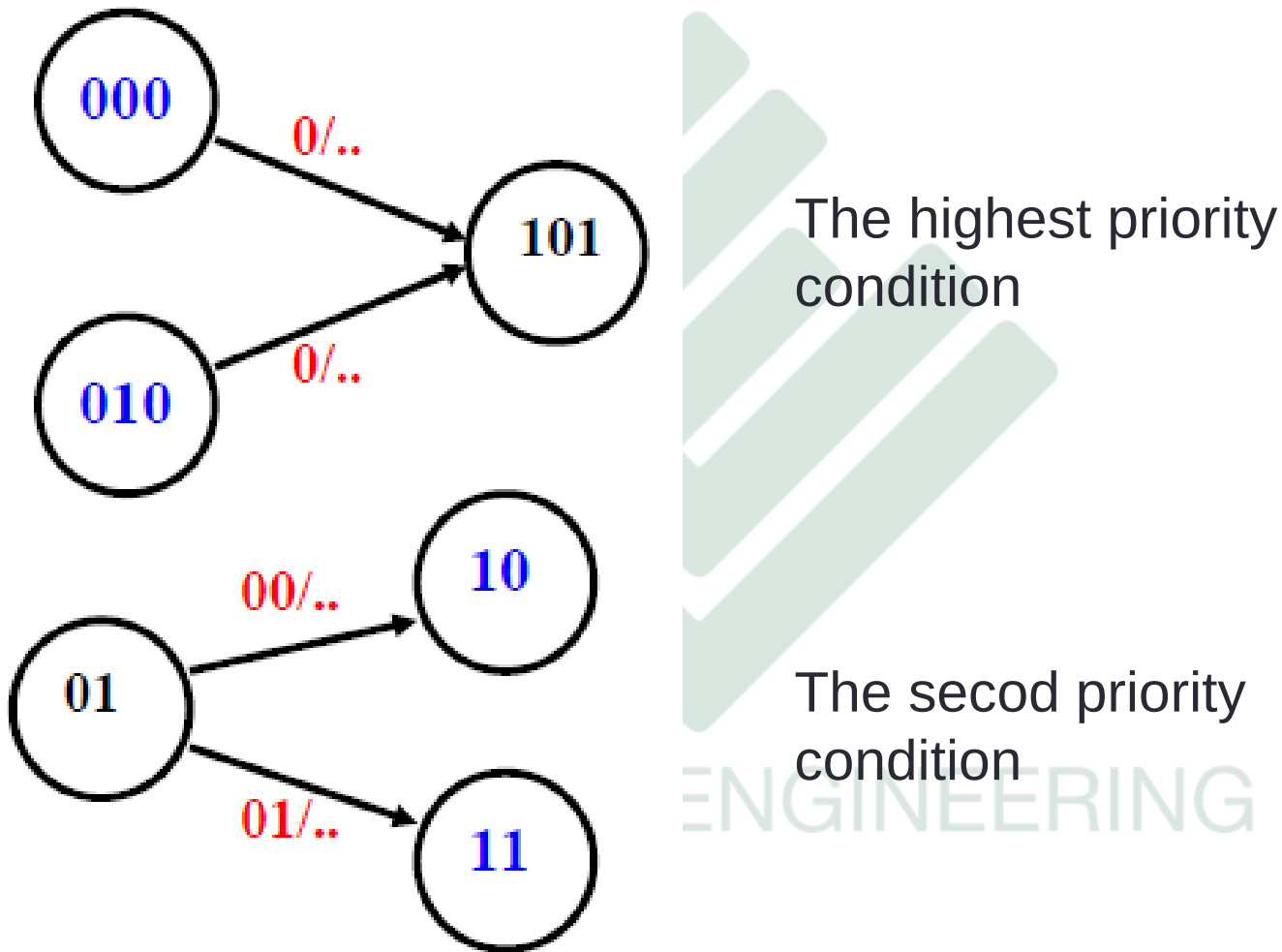
Priority 1: (s_1, s_2)
 Priority 2: (s_1, s_2)
 Priority 3: (s_0, s_1),
 (s_2, s_3)

Adjacency priorities



Possible encoding

Prioritized adjacency (cont.)



Hot-one encoding

- Hot-one encoding is a redundant encoding with one flip-flop per state having value 1 and others having value 0.
- Hot-one encoding is too expensive for FSMs with large number of states. It is used only in small FSMs.

Encoding example

- **Example:** State encodings for modulo-3 counter.
- **Problem:** Given the up/down, modulo-3 counter which was specified by the minimal next-state/output in the below table.
Find the encoding method that will minimize the cost and delay of the counter logic.

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

Encoding A = Minimum-bit-change

Encoding B = prioritized adjacency

Encoding C = Hot-one encoding

Encoding example (cont.)

➤ **Example:** State encodings for modulo-3 counter.

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

Encoding A = Minimum-bit-change

Encoding B = Prioritized adjacency

Encoding C = Hot-one encoding

Cost/delay estimation for encoding A

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

STATE	ENCODING A
	$Q_1 Q_0$
S_0	0 0
S_1	0 1
S_2	1 0

Next-state map

$Q_1 Q_0$	CD			
	00	01	11	10
00	0 0	0 0	1 0	0 1
01	0 1	0 1	0 0	1 0
11	X X	X X	X X	X X
10	1 0	1 0	0 1	0 0

$Q_1(next), Q_0(next)$

Output map

$Q_1 Q_0$	CD			
	00	01	11	10
00	0	0	1	0
01	0	0	0	0
11	X	X	X	X
10	0	0	0	1

Y

$$Q_0(next) = Q_0 C' + Q_1 C D + Q_1' Q_0' C D'$$

$$Q_1(next) = Q_1 C' + Q_0 C D' + Q_1' Q_0' C D$$

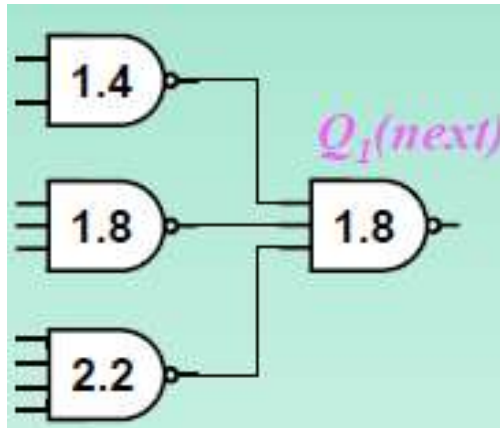
$$Y = Q_1 C D' + Q_1' Q_0' C D$$

Cost/delay estimation for encoding A

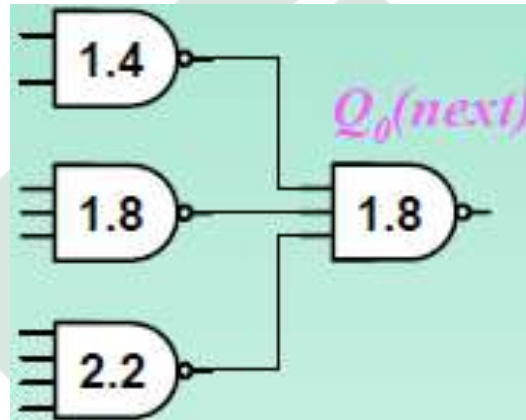
$$Q_1(next) = Q_1C' + Q_0CD' + Q_1'Q_0'CD$$

$$Q_0(next) = Q_0C' + Q_1CD + Q_1'Q_0'CD'$$

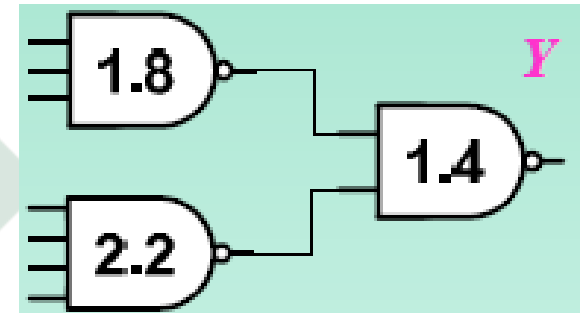
$$Y = Q_1CD' + Q_1'Q_0'CD$$



Cost (Q_1) = 24
Delay (Q_1) = 4.0



Cost (Q_0) = 24
Delay (Q_0) = 4.0



Cost (Y) = 18
Delay (Y) = 3.6

Total cost = 66

Max. input delay = 4.0ns

Max. output delay = 3.6ns

Note: each of these equation has been implemented two-level logic networks of NAND/NOR gates

Cost/delay estimation for encoding B

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

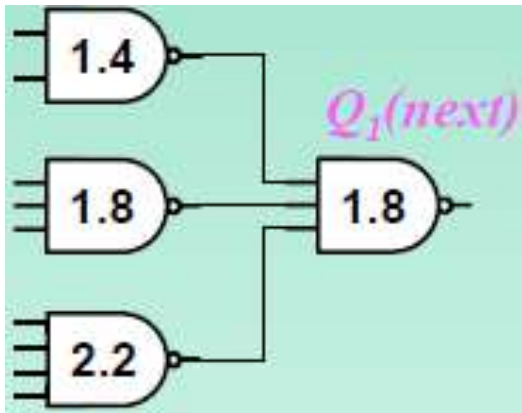
STATE	ENCODING B Q_1Q_0
S_0	0 1
S_1	0 0
S_2	1 0

Next-state map

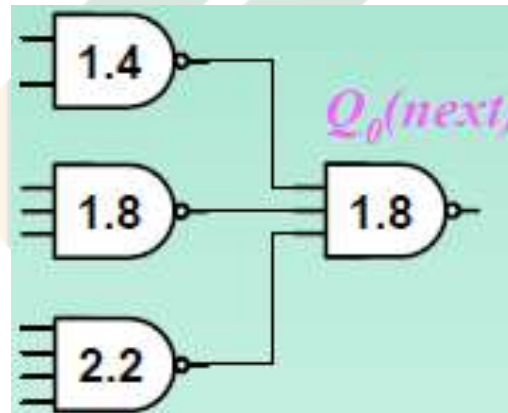
Output map

ER ENC

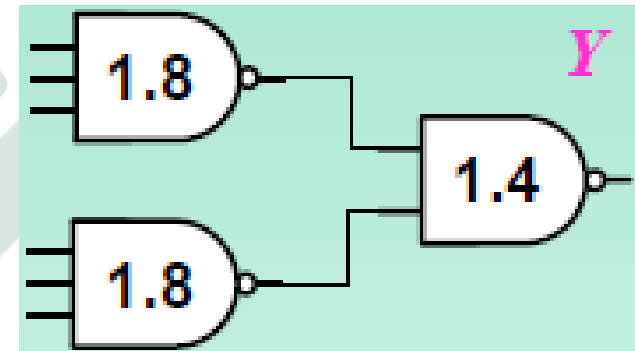
Cost/delay estimation for encoding B



Cost (Q1) = 24
Delay (Q1) = 4.0



Cost (Q0) = 24
Delay (Q0) = 4.0



Cost (Y) = 16
Delay (Y) = 3.2

Total cost = 64
Max. input delay = 4.0ns
Max. output delay = 3.2ns

COMPUTER ENGINEERING

Cost/delay estimation for encoding C

$Q_2Q_1Q_0$		CD			
		00	01	11	10
000		X X X	X X X	X X X	X X X
001		0 0 1	0 0 1	1 0 0	0 1 0
011		X X X	X X X	X X X	X X X
010		0 1 0	0 1 0	0 0 1	1 0 0
100		1 0 0	1 0 0	0 1 0	0 0 1
101		X X X	X X X	X X X	X X X
111		X X X	X X X	X X X	X X X
110		X X X	X X X	X X X	X X X

$Q_2(next), Q_1(next), Q_0(next)$

Next-state map



$Q_2Q_1Q_0$		CD			
		00	01	11	10
000		X	X	X	X
001		0	0	1	0
011		X	X	X	X
010		0	0	0	0
100		0	0	0	1
101		X	X	X	X
111		X	X	X	X
110		X	X	X	X

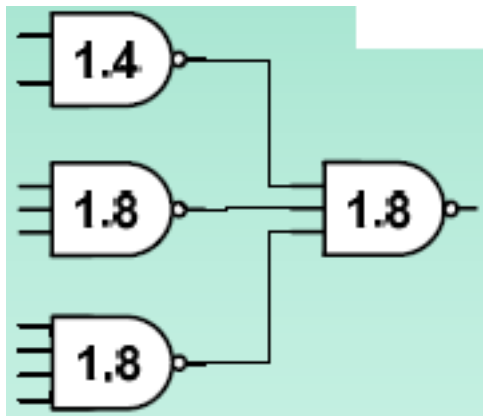
Y

Output map



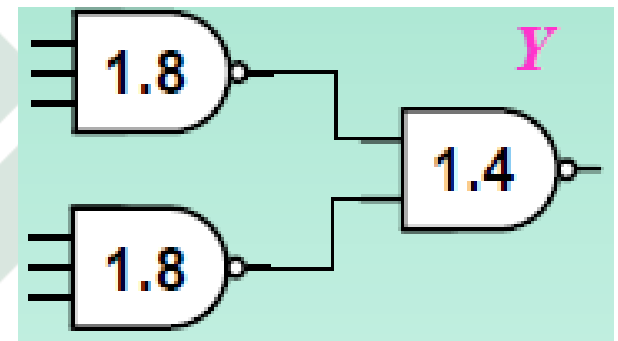
Note: 5 variable K-maps

Cost/delay estimation for encoding C



$Q_0(\text{next})$
 $Q_1(\text{next})$
 $Q_2(\text{next})$

Cost (Q_0) = Cost (Q_1) = Cost (Q_2) = 22
 Delay (Q_0) = Delay (Q_1) = Delay (Q_2) = 3.6



Cost (Y) = 16
 Delay (Y) = 3.2

COMPUTER ENGINEERING

Total cost = 82

Max. input delay = 3.6ns

Max. output delay = 3.2ns

Choice of memory elements

- Select the type of flip-flop from: D, SR, JK, T
- SR and JK reduce the cost of input logic; but require twice as many connections.
- D and T require single connection per flip-flop.
- T is suitable for *counting functions* since it changes output value when its input is 1.

COMPUTER ENGINEERING

Modulo-3 counter implementation with different flip-flops

- **Example:** Flip-flop selection.
- **Problem:** Given the modulo-3 counter with **Encoding A**, as specified below, select the type of flip-flop that will minimize the cost and/or delay of input logic.

		CD			
		00	01	11	10
Q_1Q_0	00	00	00	10	01
	01	01	01	00	10
	11	XX	XX	XX	XX
	10	10	10	01	00

$Q1(next), Q0(next)$

Next-state table for encoding A

$Q (present)$	$Q (next)$	S R		J K		T	D
0	0	0	X	0	X	0	0
0	1	1	0	1	X	1	1
1	0	0	1	X	1	1	0
1	1	X	0	X	0	0	1

Flip-flop excitation table

Modulo-3 counter implementation with different flip-flops

$Q_1 Q_0$ \ CD					
		00	01	11	10
00		0 X 0 X	0 X 0 X	1 0 0 X	0 X 1 0
01		0 X X 0	0 X X 0	0 X 0 1	1 0 0 1
11		X X X X	X X X X	X X X X	X X X X
10		X 0 0 X	X 0 0 X	0 1 1 0	0 1 0 X

S_1, R_1, S_0, R_0

$$S_1 = Q_0 CD' + Q_1' Q_0' CD \quad (\text{cost} = 18, \text{delay} = 3.6)$$

$$R_1 = Q_1 C = (Q_1' + C')' \quad (\text{cost} = 4, \text{delay} = 1.4)$$

$$S_0 = Q_1 CD + Q_1' Q_0' CD' \quad (\text{cost} = 18, \text{delay} = 3.6)$$

$$R_0 = Q_0 C = (Q_0' + C')' \quad (\text{cost} = 4, \text{delay} = 1.4)$$

Modulo-3 counter implementation with different flip-flops

CD Q ₁ Q ₀		CD			
		00	01	11	10
00		0 X 0 X	0 X 0 X	1 X 0 X	0 X 1 X
01		0 X X 0	0 X X 0	0 X X 1	1 X X 1
11		X X X X	X X X X	X X X X	X X X X
10		X 0 0 X	X 0 0 X	X 1 1 X	X 1 0 X

J₁, K₁, J₀, K₀

$$\begin{aligned}
 J_1 &= Q_0 CD' + Q_0' CD = (C' + Q_0 D + Q_0 D')' & (\text{cost} = 12, \text{delay} = 2.4) \\
 K_1 &= C & (\text{cost} = 0, \text{delay} = 0) \\
 J_0 &= Q_1 CD + Q_1' CD' = (C' + Q_1 D' + Q_1 D)' & (\text{cost} = 12, \text{delay} = 2.4) \\
 K_0 &= C & (\text{cost} = 0, \text{delay} = 0)
 \end{aligned}$$

Modulo-3 counter implementation with different flip-flops

Q_1Q_0 \ CD					
		00	01	11	10
00	0	0	0	0	1
01	0	0	0	0	1
11	X	X	X	X	X
10	0	0	0	1	0

T_1, T_0

$$T_1 = Q_1C + Q_0'CD + Q_0CD' \quad (\text{cost} = 22, \text{delay} = 3.6)$$

$$T_0 = Q_0C + Q_1CD + Q_1'CD' \quad (\text{cost} = 22, \text{delay} = 3.6)$$

Modulo-3 counter implementation with different flip-flops

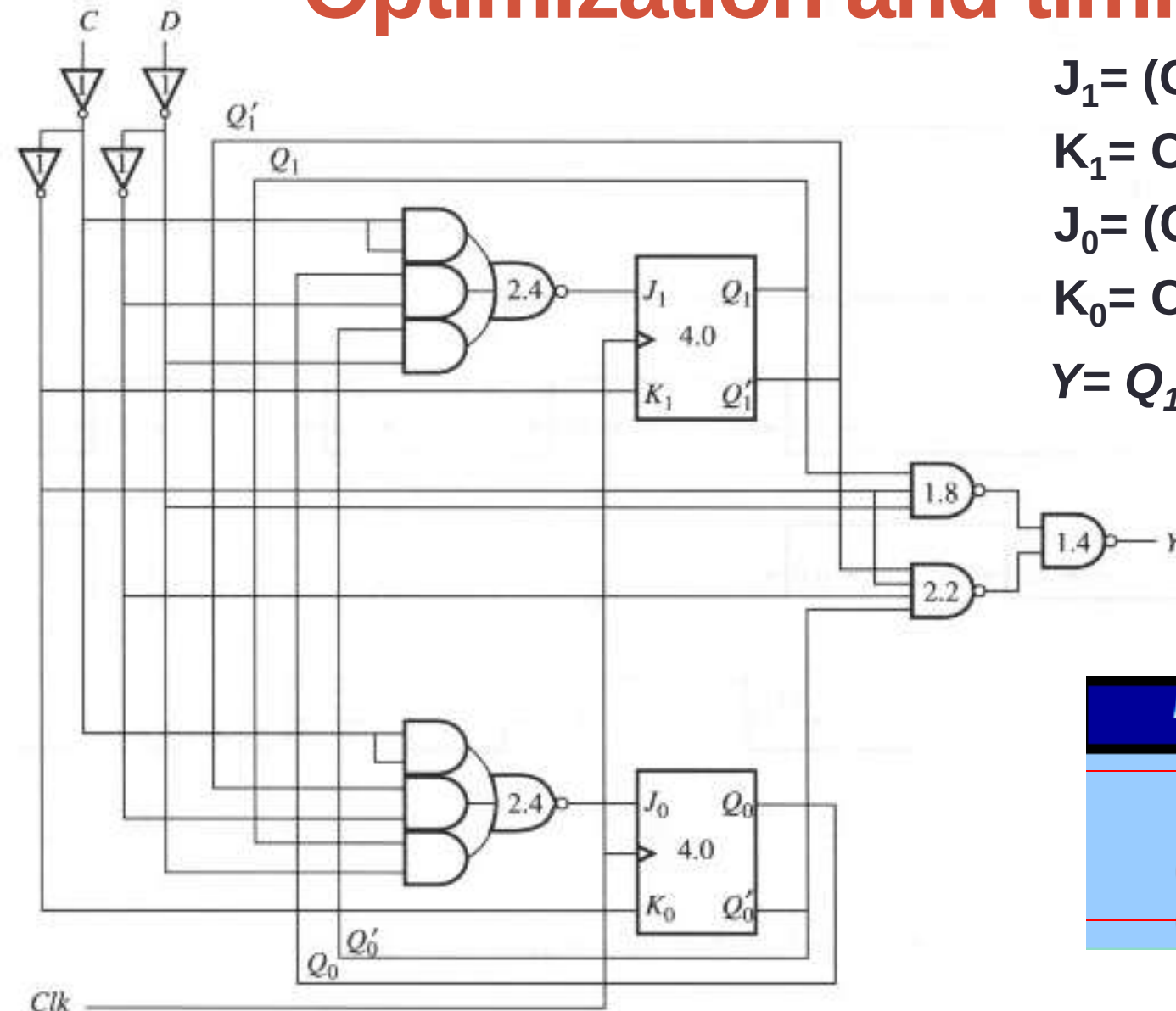
CD		Q ₁ Q ₀			
		00	01	11	10
00		00	00	10	01
01		01	01	00	10
11		X	X	X	X
10		10	10	01	00

D₁, D₀

$$D_1 = Q_1C' + Q_0CD' + Q_1'Q_0'CD \quad (\text{cost} = 24, \text{delay} = 4.0)$$

$$D_0 = Q_0C' + Q_1CD + Q_1'Q_0'CD' \quad (\text{cost} = 24, \text{delay} = 4.0)$$

Optimization and timing



$$J_1 = (C' + Q_0 D + Q_0 D')$$

$$K_1 = C$$

$$J_0 = (C' + Q_1 D' + Q_1 D')$$

$$K_0 = C$$

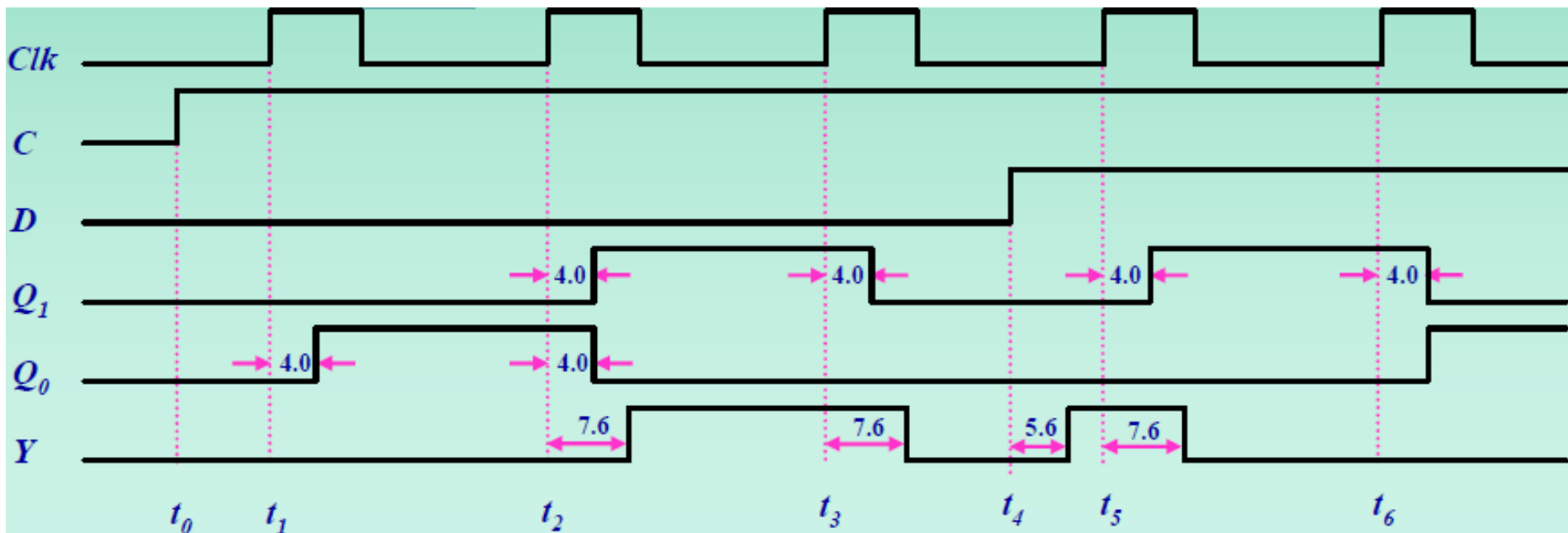
$$Y = Q_1 C D' + Q_1' Q_0' C D$$

Input-output delays		
C, D	to Clk	5.4
Clk	to Y	7.6
C, D	to Y	5.6
Clk	to Clk	7.4

Delay table

Logic schematic

Optimization and timing (cont.)



Timing diagram

COMPUTER ENGINEERING

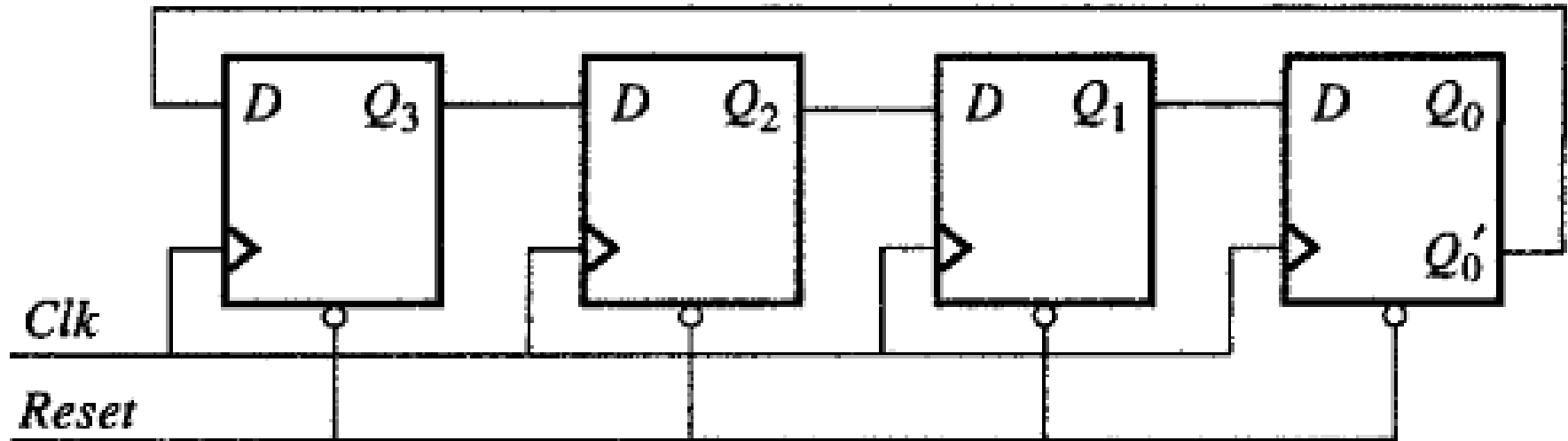
Chapter Summary

- **We introduced memory elements**
 - Latches (asynchronous)
 - Flip Flip-flops (synchronous)
- **We presented several ways to describe memory elements (flip-flops)**
 - Characteristic tables
 - Characteristic equations
 - State diagrams
 - Timing diagrams
- **We described procedures for sequential logic**
 - Analysis
 - Synthesis (include design) with
 - ❑ FSM capture
 - ❑ state minimization
 - ❑ state encoding
 - ❑ optimization and timing
- **We defined the concept of a FSM**

Bài tập 1

➤ Cho sơ đồ mạch sau:

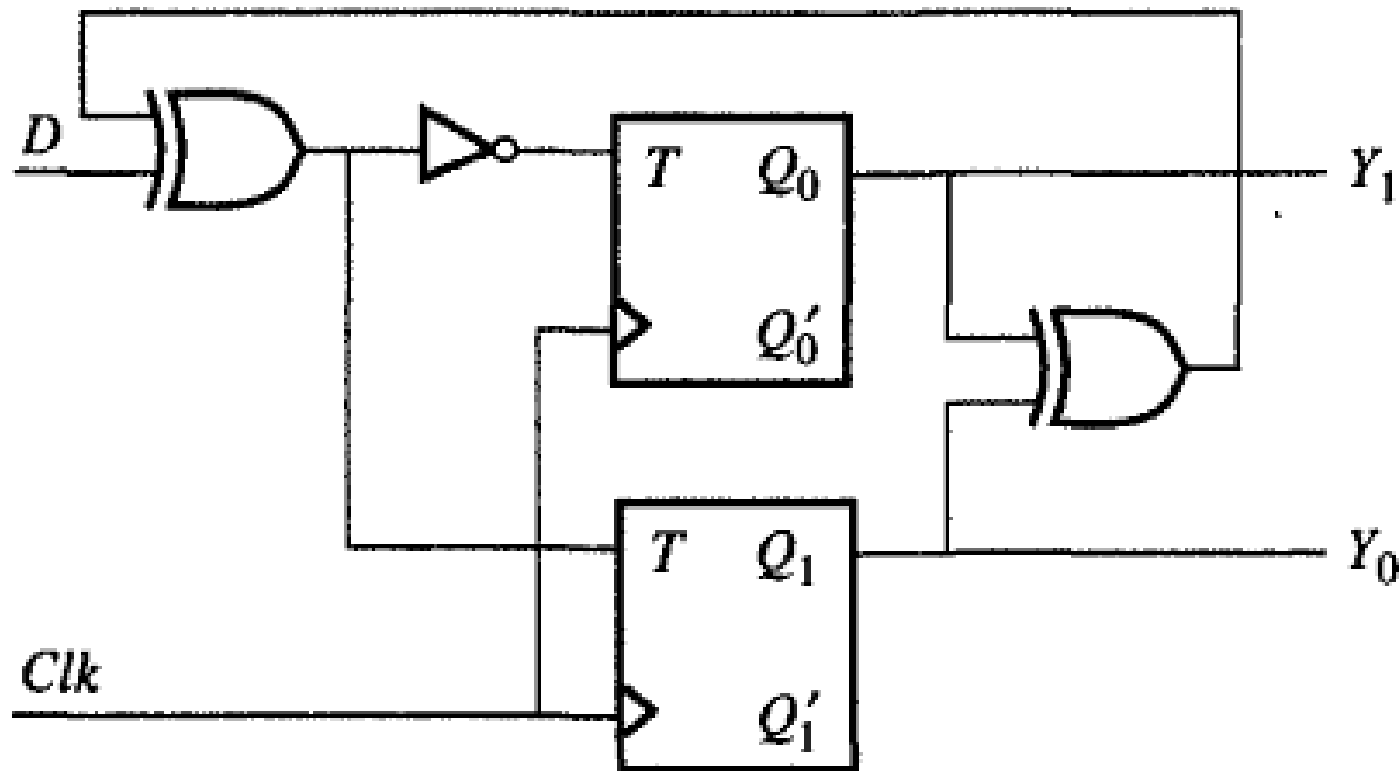
- Viết công thức cho các D và các trạng thái Q kế tiếp.
- Lập bảng trạng thái
- Vẽ sơ đồ trạng thái.



Bài tập 2

➤ Cho sơ đồ mạch sau:

- Viết công thức cho các T.
- Lập bảng trạng thái
- Vẽ sơ đồ chuyển trạng thái.



Bài tập 3

➤ Tối thiểu hóa các trạng thái, cho bảng trạng thái sau:

- Sử dụng cách phân hoạch trạng thái.
- Sử dụng bảng quan hệ (kéo theo).

Present State	NEXT STATE	
	X = 0	X = 1
S0	S0/1	S4/0
S1	S0/0	S4/0
S2	S1/0	S5/0
S3	S1/0	S5/0
S4	S2/0	S6/1
S5	S2/0	S6/1
S6	S3/0	S7/1
S7	S3/0	S7/1

Bài tập 4

➤ Tối thiểu hóa các trạng thái, cho bảng trạng thái sau:

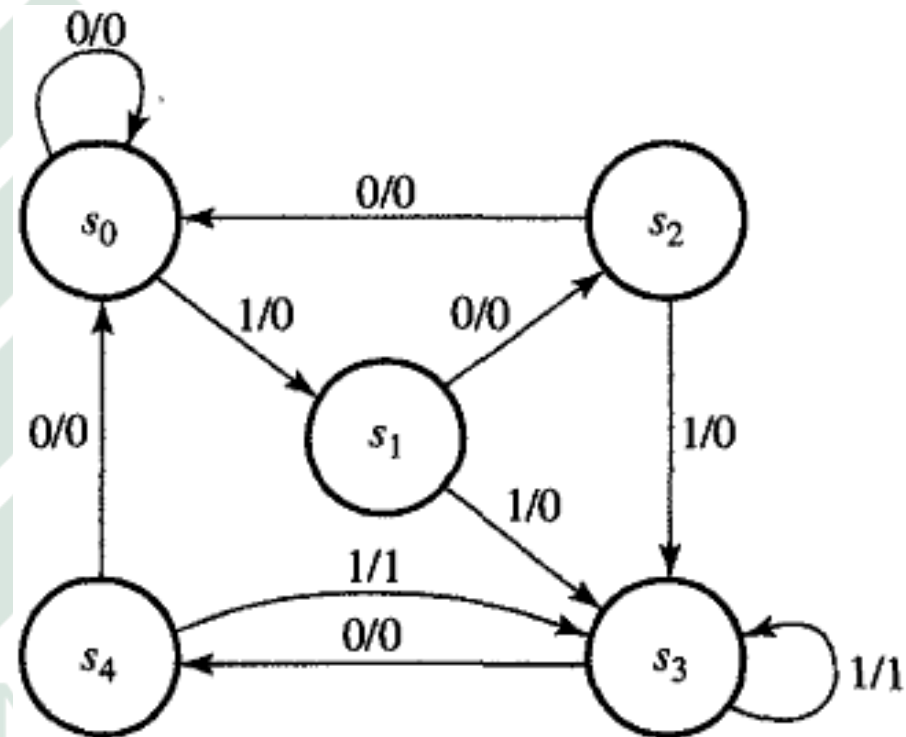
- Sử dụng cách phân hoạch trạng thái.
- Sử dụng bảng quan hệ (kéo theo).

Present State	NEXT STATE	
	X = 0	X = 1
S0	S0/1	S5/0
S1	S0/1	S5/0
S2	S1/0	S5/0
S3	S0/0	S6/0
S4	S2/0	S6/0
S5	S2/1	S6/1
S6	S3/1	S7/1
S7	S3/1	S7/1
S8	S4/1	S8/1
S9	S4/1	S9/1

Bài tập 5

➤ Giải mã trạng thái cho lược đồ quan hệ sau:

- Minimum-bit-change
- Prioritized-adjacency
- Hot-one-encoding



Bài tập 6

Thiết kế hệ tuần tự có 1 ngõ vào (X) và 1 ngõ ra Z.

Ngõ ra Z bằng 1 khi ngõ vào X là chuỗi 010 hoặc 110

Ví dụ:

```
X = 0010 0110 1100 1010 0011 ...  
Z = 0001 0001 0010 0101 0000 ..
```

COMPUTER ENGINEERING